

ĐẠI HỌC BÁCH KHOA HÀ NỘI

Khoa Toán - Tin

BÁO CÁO CUỐI KỲ
KỸ THUẬT LẬP TRÌNH
MI3310

CHỦ ĐỀ:
QUẢN LÝ SINH VIÊN VÀ ĐIỂM SỐ

Mã lớp: 169313

Giảng viên hướng dẫn: TS. Vũ Thành Nam

NHÓM 2

Họ và tên	Mã số sinh viên
Đào Văn Tuyến	202419014
Lê Nguyễn Hữu Nghĩa	202418954

HÀ NỘI, 6/2026

LỜI NÓI ĐẦU

Học phần Kỹ thuật lập trình là một học phần nền tảng trong chương trình đào tạo của sinh viên khối kỹ thuật. Môn học không chỉ yêu cầu người học biết viết lệnh theo cú pháp của một ngôn ngữ lập trình, mà còn đòi hỏi khả năng phân tích bài toán, thiết kế dữ liệu, chia chương trình thành các module có trách nhiệm rõ ràng, kiểm soát lỗi đầu vào và kiểm thử sản phẩm sau khi hoàn thiện. Những yêu cầu này đặc biệt quan trọng vì một chương trình có thể chạy được chưa chắc đã là một chương trình tốt; chương trình tốt cần có cấu trúc dễ hiểu, dễ sửa, dữ liệu được quản lý ổn định và kết quả xử lý có thể kiểm chứng.

Để tổng kết kiến thức đã học trong học phần, nhóm lựa chọn đề tài “*Ứng dụng quản lý sinh viên và điểm số*”. Đây là một bài toán gần gũi với môi trường đại học, có nghiệp vụ đủ rõ ràng và phù hợp với phạm vi cuối kỳ. Chương trình hỗ trợ quản lý danh sách sinh viên, danh mục môn học, lớp học phần, nhập và cập nhật điểm theo từng môn, tính điểm trung bình hệ 10, quy đổi GPA hệ 4, xếp loại học lực, tìm kiếm bảng điểm cá nhân, in bảng điểm lớp học phần và lập danh sách thi có cột ký tên. Các chức năng này tạo thành một hệ thống nhỏ nhưng có luồng dữ liệu hoàn chỉnh từ nhập liệu, xử lý, lưu trữ đến báo cáo.

Trong quá trình thực hiện, nhóm sử dụng ngôn ngữ C++ và tự cài đặt danh sách liên kết đơn để lưu trữ các tập dữ liệu chính. Nhóm không sử dụng các cấu trúc dữ liệu nâng cao hoặc thuật toán dựng sẵn như `vector`, `list`, `map`, `queue`, `stack` hay `sort`. Thay vào đó, các thao tác tìm kiếm, thêm, xóa, cập nhật và sắp xếp được triển khai bằng con trỏ và duyệt tuần tự trên danh sách liên kết. Dữ liệu vào ra được lưu trong các file văn bản có cấu trúc, giúp chương trình có khả năng giữ dữ liệu qua nhiều lần chạy.

Bên cạnh giao diện dòng lệnh, nhóm bổ sung chức năng xuất giao diện Dashboard dạng HTML từ chính dữ liệu của chương trình. Giao diện này đóng vai trò minh họa trực quan cho phần quản trị sinh viên và bảng điểm, đồng thời giúp báo cáo có phần trình bày sinh động hơn. Toàn bộ sản phẩm được đóng gói kèm mã nguồn, dữ liệu mẫu, file kiểm thử, ảnh chụp kết quả chạy và file LaTeX gốc để có thể chỉnh sửa hoặc biên dịch lại khi cần.

Báo cáo được trình bày theo hướng mô tả đầy đủ quá trình làm sản phẩm: giới thiệu đề tài, thiết kế cấu trúc dữ liệu và tệp tin, tổ chức chương trình, thiết kế giao diện, kết quả thực hiện, kiểm thử và tổng kết. Nhóm đã cố gắng trình bày nội dung rõ ràng, cân đối, có bảng biểu và hình ảnh minh họa để thể hiện đúng quá trình xây dựng chương trình. Do thời gian thực hiện và kinh nghiệm còn hạn chế, sản phẩm chắc chắn vẫn còn điểm có thể hoàn thiện thêm. Nhóm kính mong nhận được góp ý của Thầy để chương trình ngày càng hoàn chỉnh hơn.

Mục lục

LỜI NÓI ĐẦU	i
1 GIỚI THIỆU ĐỀ TÀI	1
1.1 Tổng quan đề tài	1
1.1.1 Mô tả đề tài	1
1.1.2 Lý do chọn đề tài	1
1.2 Khảo sát nghiệp vụ thực tế	2
1.3 Mục tiêu và phạm vi đề tài	3
1.3.1 Mục tiêu	3
1.3.2 Phạm vi đề tài	3
1.4 Tổng thể chức năng hệ thống	4
1.5 Kết quả dự kiến	5
2 THIẾT KẾ CẤU TRÚC DỮ LIỆU VÀ TẬP TIN	6
2.1 Kỹ thuật thiết kế cấu trúc dữ liệu	6
2.1.1 Lựa chọn cấu trúc dữ liệu: Danh sách liên kết đơn	6
2.1.2 Sơ đồ cấu trúc danh sách liên kết đơn	7
2.1.3 Cấu trúc lớp Node và LinkedList	7
2.1.4 Thiết kế các lớp đối tượng dữ liệu	8
2.1.5 Quan hệ giữa các lớp dữ liệu	10
2.2 Thiết kế cấu trúc lưu trữ file dữ liệu	11
2.2.1 Yêu cầu lưu trữ	11
2.2.2 Định dạng file students.txt	11
2.2.3 Định dạng file courses.txt	11
2.2.4 Định dạng file sections.txt	12
2.2.5 Định dạng file grades.txt	12
2.2.6 Tổng hợp các file dữ liệu	13
2.2.7 Cơ chế đọc file	13
2.2.8 Cơ chế ghi file	14
3 CẤU TRÚC CHƯƠNG TRÌNH VÀ CÁC MODULE CHỨC NĂNG	15
3.1 Sơ đồ tổ chức cấu trúc chương trình	15

3.2	Mô tả chi tiết các module	16
3.2.1	Module main.cpp	16
3.2.2	Module Models.h	16
3.2.3	Module LinkedList.h	17
3.2.4	Module DataStore.h/cpp	18
3.2.5	Module Services.h/cpp	18
3.2.6	Module GuiGenerator.h/cpp	19
3.2.7	Module Ui.h/cpp	20
3.2.8	Module kiểm thử test_runner.cpp	20
3.3	Quy trình vận hành hệ thống	20
3.3.1	Luồng khởi động	21
3.3.2	Luồng xử lý theo menu	21
3.4	Các thuật toán chính	22
3.4.1	Tìm kiếm tuyến tính	22
3.4.2	Sắp xếp nổi bọt	22
3.4.3	Tính GPA có trọng số tín chỉ	23
3.4.4	Kiểm tra dữ liệu nhập điểm	23
3.5	Kỹ thuật đặt tên và trình bày mã nguồn	23
4	THIẾT KẾ VÀ HIỆN THỰC HÓA GIAO DIỆN NGƯỜI DÙNG	25
4.1	Mục tiêu thiết kế giao diện	25
4.2	Giao diện dòng lệnh CLI	25
4.2.1	Nguyên tắc trình bày bảng trong CLI	26
4.3	Giao diện Dashboard HTML	26
4.3.1	Lý do chọn Dashboard HTML	26
4.3.2	Bố cục tổng thể Dashboard	27
4.3.3	Giao diện form nhập điểm	28
4.3.4	Giao diện báo cáo in danh sách thi	28
4.4	Hiện thực hóa chức năng xuất GUI	29
4.5	Đánh giá giao diện	30
5	KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ	31
5.1	Môi trường biên dịch và chạy chương trình	31
5.2	Kết quả chạy các chức năng chính	31
5.2.1	Hiển thị danh sách sinh viên	31
5.2.2	Tìm kiếm bảng điểm cá nhân	32
5.2.3	Bảng điểm lớp học phần	33
5.2.4	Nhập và cập nhật điểm	33
5.2.5	Sắp xếp danh sách sinh viên	34
5.2.6	Báo cáo in danh sách thi	35

5.2.7	Xuất giao diện Dashboard GUI	35
5.3	Kiểm thử tự động	36
5.3.1	Mục tiêu kiểm thử	36
5.3.2	Kết quả kiểm thử	37
5.4	Kiểm thử thủ công qua giao diện	37
5.5	Đánh giá chất lượng chương trình	38
6	TỔNG KẾT	39
6.1	Các kỹ thuật đã vận dụng	39
6.2	Mức độ đáp ứng yêu cầu đề bài	40
6.3	Hạn chế và hướng phát triển	40
6.4	Kết luận	41
	TÀI LIỆU THAM KHẢO	42
A	PHÂN CÔNG CÔNG VIỆC	43
B	CẤU TRÚC THƯ MỤC DỰ ÁN	44
B.1	Cây thư mục bàn giao	44
B.2	Mô tả các thư mục	44
C	PHỤ LỤC MÃ NGUỒN CHÍNH	46
C.1	File main.cpp	46
C.2	File Models.h	46
C.3	File LinkedList.h	47
C.4	Các khai báo hàm nghiệp vụ trong Services.h	50
D	PHỤ LỤC MÔ TẢ CÁC HÀM XỬ LÝ TÁC NGHIỆP CHÍNH	52
D.1	Nhóm hàm tính điểm	52
D.2	Nhóm hàm quản lý điểm	52
D.3	Nhóm hàm báo cáo	53
E	PHỤ LỤC TRÍCH MÃ NGUỒN XỬ LÝ FILE VÀ GUI	54
E.1	Trích mã nguồn đọc/ghi file	54
E.2	Trích mã nguồn xử lý nghiệp vụ	58
E.3	Trích mã nguồn xuất Dashboard GUI	63
F	PHỤ LỤC KIỂM THỬ	67
F.1	File test_runner.cpp	67
F.2	Hướng dẫn chạy lại kiểm thử	68

Danh sách hình vẽ

2.1	Cấu trúc danh sách liên kết đơn tự cài đặt	7
2.2	Mô hình quan hệ dữ liệu của hệ thống	10
3.1	Sơ đồ phụ thuộc giữa các module C++	16
3.2	Luồng xử lý tổng thể của chương trình	20
3.3	Quy trình kiểm tra dữ liệu khi nhập điểm	23
4.1	Giao diện menu chính của chương trình CLI	26
4.2	Giao diện Dashboard HTML của hệ thống	27
4.3	Thiết kế giao diện form nhập/cập nhật điểm	28
4.4	Thiết kế giao diện báo cáo in danh sách thi	29
4.5	Kết quả chạy chức năng xuất Dashboard GUI	30
5.1	Kết quả hiển thị danh sách sinh viên	32
5.2	Kết quả tìm kiếm và hiển thị bảng điểm cá nhân	32
5.3	Kết quả in bảng điểm lớp học phần	33
5.4	Kết quả nhập/cập nhật điểm hợp lệ	34
5.5	Kết quả sắp xếp danh sách sinh viên theo tên	34
5.6	Báo cáo in để nộp và ký danh sách thi	35
5.7	Kết quả xuất giao diện Dashboard HTML	36
5.8	Kết quả kiểm thử tự động	37
B.1	Cây thư mục bàn giao của dự án	44

Danh sách bảng

1.1	Khảo sát các khái niệm nghiệp vụ thực tế	2
1.2	Phạm vi giới hạn của đề tài	4
1.3	Danh sách tổng thể chức năng hệ thống	4
2.1	So sánh mảng tĩnh, mảng động và danh sách liên kết đơn	6
2.2	Mô tả các phương thức chính của lớp LinkedList	8
2.3	Thuộc tính của lớp Student	9
2.4	Thuộc tính của lớp Course	9
2.5	Thuộc tính của lớp ClassSection	9
2.6	Thuộc tính của lớp Grade	10
2.7	Cấu trúc các trường trong file students.txt	11
2.8	Cấu trúc các trường trong file courses.txt	12
2.9	Cấu trúc các trường trong file sections.txt	12
2.10	Cấu trúc các trường trong file grades.txt	13
2.11	Tổng hợp các file dữ liệu của hệ thống	13
3.1	Danh sách các module và vai trò trong hệ thống	15
3.2	Các hàm nghiệp vụ chính trong Services.cpp	19
3.3	Workflow chi tiết theo từng lựa chọn menu	21
3.4	Quy ước đặt tên trong mã nguồn	24
4.1	Nguyên tắc thiết kế bảng CLI	26
4.2	Cấu trúc bố cục Dashboard HTML	27
4.3	Đánh giá ưu điểm và hạn chế của giao diện	30
5.1	Các tình huống kiểm thử tự động	36
5.2	Tổng hợp kiểm thử thủ công qua menu	38
6.1	Tổng kết các kỹ thuật đã vận dụng	39
6.2	Đối chiếu sản phẩm với yêu cầu đề bài	40
A.1	Bảng phân công nhiệm vụ và khối lượng công việc	43
B.1	Mô tả các thư mục trong gói nộp	44
D.1	Mô tả nhóm hàm tính điểm	52

D.2	Mô tả nhóm hàm quản lý điểm	52
D.3	Mô tả nhóm hàm báo cáo trong Ui.cpp	53

CHƯƠNG 1.

GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan đề tài

1.1.1 Mô tả đề tài

Đề tài hướng tới việc xây dựng một chương trình quản lý sinh viên và điểm số bằng C++. Người dùng thao tác thông qua menu để thực hiện các nghiệp vụ cơ bản của một hệ thống quản lý đào tạo quy mô nhỏ: xem danh sách sinh viên, thêm sinh viên, cập nhật thông tin sinh viên, xóa sinh viên, xem danh mục môn học, nhập hoặc cập nhật điểm theo mã sinh viên và mã môn học, tìm kiếm bảng điểm của một sinh viên, in bảng điểm lớp học phần, sắp xếp danh sách theo MSSV hoặc họ tên và lập báo cáo danh sách thi có ô ký tên.

Hệ thống được thiết kế theo hướng đơn giản nhưng đầy đủ vòng đời dữ liệu. Khi chương trình khởi động, dữ liệu được nạp từ các file văn bản trong thư mục **data**. Trong lúc chạy, dữ liệu được lưu trong các danh sách liên kết đơn tự cài đặt. Khi người dùng chọn kết thúc, chương trình ghi lại dữ liệu xuống file. Với cách tổ chức này, chương trình không phụ thuộc vào cơ sở dữ liệu ngoài nhưng vẫn bảo đảm dữ liệu không bị mất sau mỗi lần chạy.

Các nhóm chức năng cốt lõi của hệ thống gồm:

- Quản lý danh mục sinh viên, môn học và lớp học phần.
- Quản lý điểm học tập theo cặp khóa MSSV và mã môn học.
- Tính điểm tổng kết hệ 10 theo công thức trọng số, quy đổi điểm hệ 4 và xếp loại học lực.
- Tìm kiếm, sắp xếp và in báo cáo theo yêu cầu của người dùng.
- Xuất giao diện Dashboard HTML để hiển thị trực quan dữ liệu sinh viên và điểm số.

1.1.2 Lý do chọn đề tài

Trong thực tế học tập ở bậc đại học, dữ liệu sinh viên và điểm số luôn cần được cập nhật thường xuyên. Nếu quản lý thủ công bằng bảng tính rời rạc, người dùng dễ gặp các lỗi như nhập sai mã sinh viên, tính điểm trung bình không nhất quán, không kiểm soát được điểm ngoài khoảng

hợp lệ hoặc khó lập báo cáo khi cần in danh sách thi. Vì vậy, bài toán quản lý sinh viên là bài toán thực tế, dễ hiểu nhưng vẫn đủ nhiều thành phần để vận dụng kiến thức của học phần Kỹ thuật lập trình.

Dưới góc độ kỹ thuật, đề tài có ưu điểm là dữ liệu có cấu trúc rõ ràng. Sinh viên, môn học, lớp học phần và điểm đều có thể mô hình hóa thành các kiểu dữ liệu riêng. Giữa các thực thể có quan hệ logic thông qua mã định danh: điểm của một sinh viên gắn với mã sinh viên, mã môn và mã lớp học phần. Từ đó, nhóm có thể thực hành thiết kế cấu trúc dữ liệu, hàm xử lý nghiệp vụ, đọc ghi file, tìm kiếm và sắp xếp. Đây là những nội dung trọng tâm của môn Kỹ thuật lập trình.

Đề tài cũng có giá trị thực hành vì sản phẩm cuối cùng có thể chạy được, có dữ liệu mẫu, có kiểm thử và có báo cáo in ra. Nhóm không chỉ viết một chương trình tính toán đơn lẻ, mà xây dựng một ứng dụng nhỏ có nhiều chức năng liên kết với nhau. Điều này giúp nhóm rèn luyện tư duy tổ chức chương trình và quy trình làm phần mềm cơ bản.

1.2 Khảo sát nghiệp vụ thực tế

Để xác định phạm vi chương trình, nhóm khảo sát các khái niệm nghiệp vụ thường gặp trong quản lý đào tạo. Các khái niệm này được chuyển hóa thành lớp dữ liệu và file lưu trữ trong chương trình.

Bảng 1.1: Khảo sát các khái niệm nghiệp vụ thực tế

Khái niệm	Định nghĩa	Ví dụ thực tế
Sinh viên	Người học có mã số sinh viên duy nhất, thuộc một lớp quản lý và có thông tin cá nhân cơ bản.	Đào Văn Tuyến - 202419014 - MI2-01.
Môn học	Đơn vị học tập có mã môn, tên môn và số tín chỉ.	MI3310 - Kỹ thuật lập trình - 3 tín chỉ.
Lớp học phần	Lớp được mở cho một môn trong một học kỳ, có giảng viên phụ trách.	MI3310-01, học kỳ 2025.2.
Điểm thành phần	Điểm quá trình, giữa kỳ và cuối kỳ của sinh viên trong một môn.	QT = 8.5, GK = 8.0, CK = 9.0.
Điểm tổng kết	Điểm hệ 10 được tính theo trọng số các điểm thành phần.	Tổng = $0.2*QT + 0.3*GK + 0.5*CK$.
GPA hệ 4	Điểm trung bình quy đổi sang thang 4 và có xét số tín chỉ.	GPA = 3.57.
Bảng điểm cá nhân	Bảng tổng hợp kết quả các môn đã có điểm của một sinh viên.	Bảng điểm của MSSV 202419014.
Bảng điểm lớp học phần	Bảng điểm của tất cả sinh viên trong một lớp học phần.	Bảng điểm lớp MI3310-01.
Danh sách thi	Báo cáo in gồm thông tin sinh viên, trạng thái đủ điều kiện và cột ký tên.	Danh sách thi cuối kỳ môn MI3310.

Từ bảng khảo sát, có thể thấy chương trình cần xử lý cả dữ liệu tĩnh và dữ liệu động. Dữ liệu tĩnh gồm danh sách môn học và lớp học phần, thường ít thay đổi. Dữ liệu động gồm sinh viên và điểm số, có thể được thêm, sửa, xóa trong quá trình sử dụng. Vì vậy, nhóm lựa chọn danh sách liên kết đơn để thuận tiện cho thao tác thêm và xóa phần tử trong phạm vi học thuật của môn học.

1.3 Mục tiêu và phạm vi đề tài

1.3.1 Mục tiêu

Mục tiêu của sản phẩm được chia thành mục tiêu nghiệp vụ và mục tiêu kỹ thuật. Về nghiệp vụ, chương trình phải đủ chức năng để quản lý sinh viên và điểm số ở mức cơ bản. Về kỹ thuật, chương trình phải thể hiện rõ việc tự cài đặt cấu trúc dữ liệu, thuật toán và kiểm thử.

- Xây dựng chương trình C++ có menu thao tác rõ ràng, cho phép người dùng chọn chức năng đến khi chọn kết thúc.
- Quản lý được danh sách sinh viên, danh sách môn học, lớp học phần và dữ liệu điểm.
- Nhập và cập nhật điểm theo MSSV, mã môn học và mã lớp học phần.
- Tính điểm tổng kết hệ 10 theo công thức trọng số, quy đổi GPA hệ 4 và xếp loại học lực.
- Tìm kiếm bảng điểm cá nhân, in bảng điểm lớp học phần và báo cáo danh sách thi.
- Tự cài đặt danh sách liên kết đơn, tìm kiếm tuyến tính và sắp xếp nổi bọt.
- Lưu trữ dữ liệu bằng file text, không phụ thuộc cơ sở dữ liệu hoặc thư viện xử lý dữ liệu nâng cao.
- Có bộ kiểm thử tự động và ảnh chụp kết quả chạy để minh chứng chương trình hoạt động.

1.3.2 Phạm vi đề tài

Phạm vi được giới hạn để bảo đảm phù hợp với thời lượng cuối kỳ và yêu cầu học phần. Chương trình không hướng tới thay thế hệ thống quản lý đào tạo chuyên nghiệp, mà tập trung vào phần lõi dữ liệu và thuật toán.

Bảng 1.2: Phạm vi giới hạn của đề tài

Tiêu chí	Mô tả
Đối tượng sử dụng	Sinh viên hoặc cán bộ lớp cần quản lý điểm, bảng điểm và danh sách thi ở quy mô lớp học phần.
Ngôn ngữ lập trình	C++ chuẩn C++11, biên dịch bằng g++.
Cấu trúc dữ liệu	Danh sách liên kết đơn tự cài đặt, không dùng <code>vector</code> , <code>list</code> , <code>map</code> , <code>queue</code> , <code>sort</code> .
Lưu trữ dữ liệu	File văn bản phân tách bằng dấu : <code>students.txt</code> , <code>courses.txt</code> , <code>sections.txt</code> , <code>grades.txt</code> .
Giao diện	Giao diện dòng lệnh dạng bảng và chức năng xuất Dashboard HTML để mở bằng trình duyệt.
Giới hạn	Chưa hỗ trợ đăng nhập nhiều vai trò, chưa đồng bộ dữ liệu qua mạng, chưa dùng cơ sở dữ liệu quan hệ.

1.4 Tổng thể chức năng hệ thống

Bảng 1.3 tổng hợp các chức năng chính. Các chức năng được chia thành nhóm quản lý danh mục, nhóm xử lý điểm, nhóm báo cáo và nhóm giao diện.

Bảng 1.3: Danh sách tổng thể chức năng hệ thống

STT	Chức năng	Nhóm	Mô tả chi tiết
1	Hiển thị sinh viên	Quản lý	In danh sách sinh viên kèm tín chỉ đã học, điểm trung bình hệ 10, GPA hệ 4 và xếp loại.
2	Thêm sinh viên	Quản lý	Nhập MSSV, họ tên, lớp, ngày sinh, giới tính, số điện thoại và email; kiểm tra trùng MSSV.
3	Cập nhật sinh viên	Quản lý	Tìm sinh viên theo MSSV và cho phép chỉnh sửa họ tên, lớp hoặc ngày sinh.
4	Xóa sinh viên	Quản lý	Xóa node sinh viên khỏi danh sách liên kết theo MSSV.
5	Hiển thị môn học	Danh mục	In mã môn, tên môn và số tín chỉ của từng môn.
6	Nhập/cập nhật điểm	Điểm số	Nhập điểm quá trình, giữa kỳ, cuối kỳ theo MSSV, mã môn, mã lớp học phần; tự tính điểm tổng.
7	Bảng điểm cá nhân	Báo cáo	Tìm kiếm sinh viên và hiển thị toàn bộ điểm các môn đã học, GPA và xếp loại.
8	Bảng điểm lớp học phần	Báo cáo	In danh sách điểm của các sinh viên thuộc một lớp học phần cụ thể.

STT	Chức năng	Nhóm	Mô tả chi tiết
9	Sắp xếp danh sách	Thuật toán	Sắp xếp sinh viên theo MSSV hoặc họ tên bằng thuật toán nổi bật tự cài đặt.
10	Danh sách thi ký tên	Báo cáo	Lập báo cáo in có thông tin sinh viên, điểm quá trình, điểm tổng, trạng thái và ô ký tên.
11	Xuất Dashboard GUI	Giao diện	Sinh file <code>gui/dashboard.html</code> từ dữ liệu hiện có để mở bằng trình duyệt.
0	Lưu và kết thúc	Hệ thống	Ghi toàn bộ dữ liệu về file text và kết thúc vòng lặp chương trình.

1.5 Kết quả dự kiến

Sau khi hoàn thiện, sản phẩm bàn giao gồm mã nguồn C++, file dữ liệu mẫu, file kiểm thử, ảnh chụp kết quả chạy, file báo cáo PDF và toàn bộ mã nguồn LaTeX. Chương trình có thể biên dịch bằng `make`, chạy bằng `./student_manager`, chạy kiểm thử bằng `make test`, và xuất Dashboard GUI bằng lựa chọn số 11 trong menu. Sản phẩm đáp ứng đúng yêu cầu bài cuối kỳ: có chương trình chạy được, có dữ liệu vào ra lưu file text, có cấu trúc dữ liệu tự cài đặt, có thuật toán tìm kiếm/sắp xếp tự viết, có kiểm thử và có báo cáo trình bày đầy đủ.

CHƯƠNG 2.

THIẾT KẾ CẤU TRÚC DỮ LIỆU VÀ TẬP TIN

2.1 Kỹ thuật thiết kế cấu trúc dữ liệu

2.1.1 Lựa chọn cấu trúc dữ liệu: Danh sách liên kết đơn

Hệ thống cần lưu trữ bốn nhóm dữ liệu chính: sinh viên, môn học, lớp học phần và điểm. Các nhóm dữ liệu này có số lượng phần tử thay đổi trong quá trình sử dụng. Người dùng có thể thêm sinh viên mới, xóa sinh viên, cập nhật điểm hoặc bổ sung bản ghi điểm. Theo yêu cầu của học phần, nhóm không dùng các cấu trúc dữ liệu nâng cao có sẵn, do đó danh sách liên kết đơn được lựa chọn làm cấu trúc lưu trữ nền tảng.

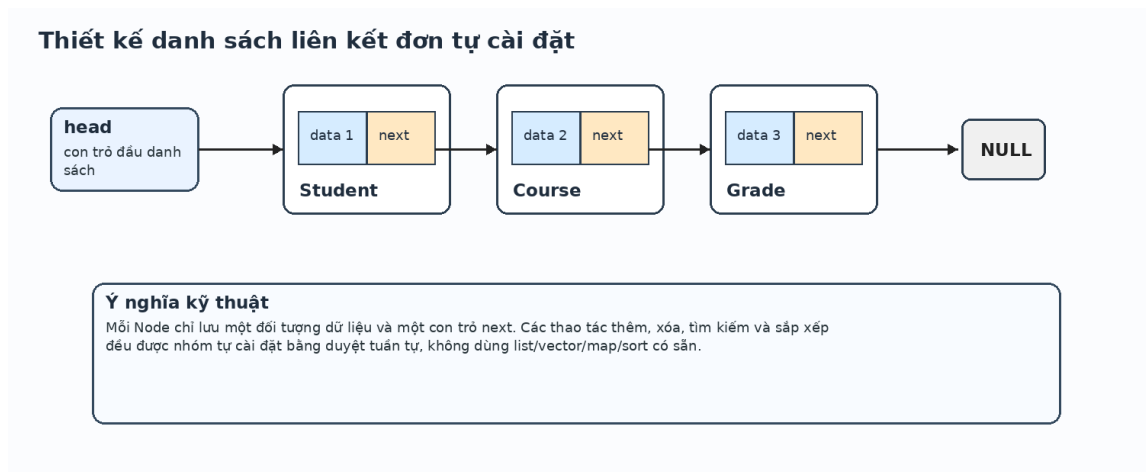
Danh sách liên kết đơn phù hợp với mục tiêu học thuật vì thể hiện rõ cách quản lý vùng nhớ động, cách nối các phần tử bằng con trỏ và cách thao tác với node. Mặc dù việc tìm kiếm trên danh sách liên kết có độ phức tạp tuyến tính, quy mô dữ liệu của đề tài ở mức lớp học phần nên chi phí này chấp nhận được. Quan trọng hơn, cấu trúc này giúp nhóm thực hành trực tiếp kiến thức con trỏ trong C++.

Bảng 2.1: So sánh mảng tĩnh, mảng động và danh sách liên kết đơn

Tiêu chí	Mảng tĩnh	Mảng động/thư viện	Danh sách liên kết đơn tự cài
Cấp phát bộ nhớ	Kích thước cố định, khó mở rộng khi vượt giới hạn.	Thường tự tăng kích thước, nhưng là cấu trúc có sẵn.	Mỗi phần tử cấp phát thành một Node riêng và nối bằng con trỏ.
Thêm phần tử	Nhanh nếu còn chỗ trống, nhưng phải đặt giới hạn trước.	Thuận tiện nhưng không phù hợp yêu cầu không dùng thư viện nâng cao.	Có thể thêm ở cuối bằng cách duyệt đến node cuối.
Xóa phần tử	Phải dịch chuyển các phần tử phía sau.	Thư viện xử lý sẵn.	Chỉ cần điều chỉnh con trỏ next.
Tìm kiếm	Duyệt tuần tự nếu không có chỉ mục.	Có thể dùng hàm có sẵn.	Duyệt tuần tự và so sánh khóa.
Ý nghĩa học thuật	Thể hiện tư duy mảng nhưng ít thao tác con trỏ.	Không thể hiện rõ cách cài đặt cấu trúc dữ liệu.	Phù hợp yêu cầu tự cài đặt và rèn luyện con trỏ.

2.1.2 Sơ đồ cấu trúc danh sách liên kết đơn

Mỗi phần tử của danh sách được gọi là một Node. Node chứa hai thành phần: trường data lưu đối tượng dữ liệu thực tế và trường next trỏ tới node kế tiếp. Node cuối cùng có next = NULL. Toàn bộ danh sách được quản lý bằng con trỏ head. Khi head = NULL, danh sách đang rỗng.



Hình 2.1: Cấu trúc danh sách liên kết đơn tự cài đặt

2.1.3 Cấu trúc lớp Node và LinkedList

Lớp LinkedList được viết dạng template để có thể dùng lại cho nhiều kiểu dữ liệu khác nhau như Student, Course, ClassSection và Grade. Việc dùng template không phải là cấu trúc dữ liệu nâng cao có sẵn, mà là kỹ thuật tổng quát hóa mã nguồn trong C++ giúp tránh lặp lại bốn lớp danh sách giống nhau.

```

1  template <class T>
2  struct Node {
3      T data;
4      Node<T>* next;
5      Node(const T& value) {
6          data = value;
7          next = 0;
8      }
9  };
10
11 template <class T>
12 class LinkedList {
13 private:
14     Node<T>* head;
15 public:
16     LinkedList() { head = 0; }
17     ~LinkedList() { clear(); }
18     Node<T>* getHead() const { return head; }
    
```

```

19  bool isEmpty() const { return head == 0; }
20  };

```

Listing 2.1: Cấu trúc Node và các hàm cơ bản của LinkedList**Bảng 2.2:** Mô tả các phương thức chính của lớp LinkedList

Phương thức	Chức năng	Độ phức tạp
<code>addBack(value)</code>	Tạo Node mới chứa dữ liệu và gắn vào cuối danh sách. Nếu danh sách rỗng thì cập nhật <code>head</code> .	$O(n)$
<code>findById(id)</code>	Duyệt từ <code>head</code> , so sánh trường <code>id</code> của từng phần tử với khóa cần tìm.	$O(n)$
<code>removeById(id)</code>	Tìm node có khóa tương ứng, sau đó nối node trước với node sau để loại bỏ node cần xóa.	$O(n)$
<code>count()</code>	Đếm số node hiện có trong danh sách.	$O(n)$
<code>isEmpty()</code>	Kiểm tra danh sách rỗng bằng con trỏ <code>head</code> .	$O(1)$
<code>clear()</code>	Duyệt toàn bộ danh sách và giải phóng từng node bằng <code>delete</code> .	$O(n)$
<code>bubbleSort(compare)</code>	Sắp xếp dữ liệu trong các node bằng thuật toán nổi bọt và hàm so sánh truyền vào.	$O(n^2)$

Điểm quan trọng trong thiết kế là lớp `LinkedList` không biết chi tiết nghiệp vụ của `Student` hay `Grade`. Lớp này chỉ cung cấp thao tác lưu trữ tổng quát. Phần nghiệp vụ như tính GPA, xếp loại hay kiểm tra mã môn được đặt ở module `Services`. Cách tách này giúp chương trình dễ mở rộng hơn.

2.1.4 Thiết kế các lớp đối tượng dữ liệu

Các lớp dữ liệu được đặt trong file `Models.h`. Mỗi lớp được định nghĩa bằng `struct` để tập trung vào lưu trữ dữ liệu, còn xử lý nghiệp vụ đặt ở module riêng. Cách làm này phù hợp với phạm vi học phần và giúp mã nguồn dễ đọc.

Lớp Student**Bảng 2.3:** Thuộc tính của lớp Student

Thuộc tính	Kiểu	Ý nghĩa	Ví dụ
id	string	Mã số sinh viên, dùng làm khóa tìm kiếm và xóa.	202419014
fullName	string	Họ tên đầy đủ của sinh viên.	Dao Van Tuyen
className	string	Lớp quản lý hoặc lớp hành chính.	MI2-01
birthDate	string	Ngày sinh định dạng YYYY-MM-DD.	2006-05-19
gender	string	Giới tính.	Nam
phone	string	Số điện thoại liên hệ.	0987654321
email	string	Email sinh viên.	tuyen.dv@example.edu.vn

Lớp Course**Bảng 2.4:** Thuộc tính của lớp Course

Thuộc tính	Kiểu	Ý nghĩa	Ví dụ
id	string	Mã môn học, dùng làm khóa định danh.	MI3310
name	string	Tên môn học.	Kỹ thuật lập trình
credits	int	Số tín chỉ dùng để tính điểm trung bình có trọng số.	3

Lớp ClassSection**Bảng 2.5:** Thuộc tính của lớp ClassSection

Thuộc tính	Kiểu	Ý nghĩa	Ví dụ
id	string	Mã lớp học phần.	MI3310-01
courseId	string	Mã môn học mà lớp học phần thuộc về.	MI3310
semester	string	Học kỳ mở lớp.	2025.2
academicYear	string	Năm học.	2025-2026
lecturer	string	Giảng viên phụ trách.	TS. Vu Thanh Nam

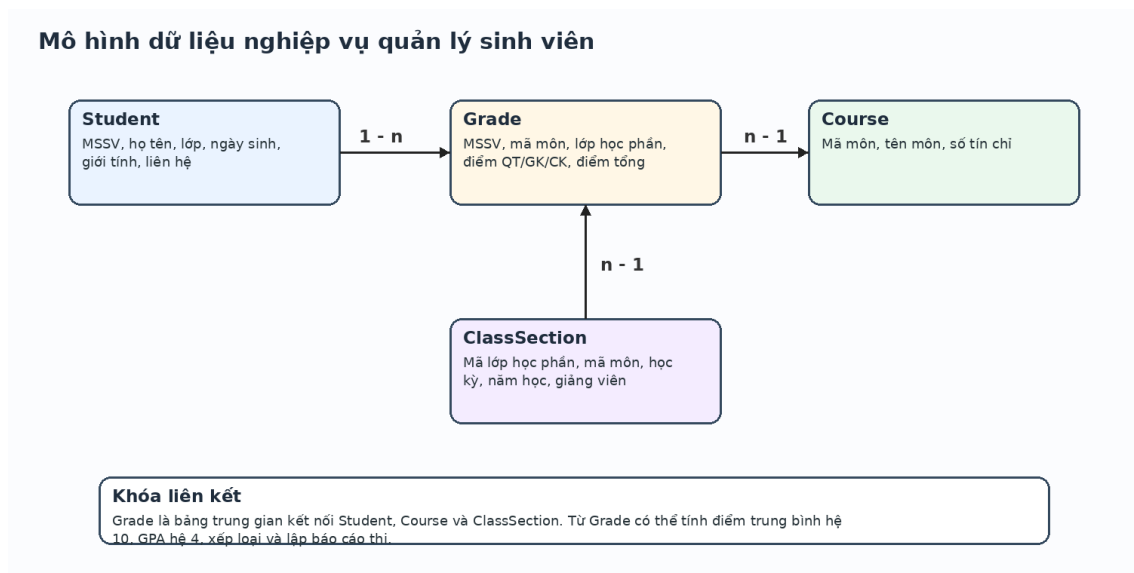
Lớp Grade

Bảng 2.6: Thuộc tính của lớp Grade

Thuộc tính	Kiểu	Ý nghĩa	Ví dụ
id	string	Mã bản ghi điểm, sinh từ MSSV và mã môn.	G_202419014_MI3310
studentId	string	Khóa liên kết tới sinh viên.	202419014
courseId	string	Khóa liên kết tới môn học.	MI3310
sectionId	string	Khóa liên kết tới lớp học phần.	MI3310-01
processScore	double	Điểm quá trình.	8.5
midtermScore	double	Điểm giữa kỳ.	8.0
finalScore	double	Điểm cuối kỳ.	9.0
total10	double	Điểm tổng kết hệ 10.	8.60

2.1.5 Quan hệ giữa các lớp dữ liệu

Quan hệ giữa các lớp dữ liệu được thể hiện qua các trường khóa. **Grade** là lớp trung gian kết nối **Student**, **Course** và **ClassSection**. Nhờ đó, từ một bản ghi điểm có thể biết sinh viên nào học môn nào, thuộc lớp học phần nào và có kết quả ra sao.



Hình 2.2: Mô hình quan hệ dữ liệu của hệ thống

Thiết kế này có ưu điểm là tránh lặp lại quá nhiều thông tin. Ví dụ trong file điểm không cần lưu tên môn hoặc tên sinh viên, chỉ cần lưu mã sinh viên và mã môn. Khi cần in báo cáo, chương trình tra cứu ngược lại qua danh sách sinh viên và danh sách môn học. Cách làm này gần với tư duy thiết kế cơ sở dữ liệu nhưng vẫn được cài đặt bằng file text và danh sách liên kết đơn.

2.2 Thiết kế cấu trúc lưu trữ file dữ liệu

2.2.1 Yêu cầu lưu trữ

Dữ liệu trong RAM sẽ mất khi chương trình kết thúc, vì vậy chương trình cần cơ chế lưu trữ bền vững. Nhóm lựa chọn file text vì dễ kiểm tra, dễ chỉnh sửa thủ công khi cần và phù hợp với yêu cầu học phần. Mỗi dòng trong file tương ứng với một đối tượng dữ liệu. Các trường được phân tách bằng dấu gạch đứng |.

Các yêu cầu lưu trữ được xác định như sau:

- Dữ liệu phải đọc được khi chương trình khởi động.
- Dữ liệu phải ghi lại được khi người dùng chọn kết thúc.
- Mỗi file chỉ lưu một loại dữ liệu để dễ quản lý.
- Nếu file chưa tồn tại, chương trình không được đột ngột mà chỉ tạo danh sách rỗng.
- Khi ghi file, các ký tự có thể phá định dạng như | hoặc xuống dòng được thay thế bằng ký tự an toàn.

2.2.2 Định dạng file students.txt

File data/students.txt lưu danh sách sinh viên. Mỗi dòng có cấu trúc:

MSSV|Ho ten|Lop|Ngay sinh|Gioi tinh|So dien thoai|Email

Bảng 2.7: Cấu trúc các trường trong file students.txt

Vị trí	Tên trường	Kiểu	Ghi chú
0	MSSV	string	Khóa chính, không để trùng khi thêm sinh viên.
1	Họ tên	string	Lưu không dấu trong dữ liệu mẫu để bảo đảm console hiển thị ổn định.
2	Lớp	string	Lớp quản lý của sinh viên.
3	Ngày sinh	string	Kiểm tra theo định dạng YYYY-MM-DD.
4	Giới tính	string	Nam hoặc Nu.
5	Số điện thoại	string	Thông tin liên hệ.
6	Email	string	Email của sinh viên.

2.2.3 Định dạng file courses.txt

File data/courses.txt lưu danh mục môn học. Mỗi dòng có cấu trúc:

```
Ma mon|Ten mon|So tin chi
```

Bảng 2.8: Cấu trúc các trường trong file courses.txt

Vị trí	Tên trường	Kiểu	Ghi chú
0	Mã môn	string	Khóa định danh môn học, ví dụ MI3310.
1	Tên môn	string	Tên đầy đủ của môn học.
2	Số tín chỉ	int	Dùng để tính điểm trung bình có trọng số.

2.2.4 Định dạng file sections.txt

File data/sections.txt lưu thông tin lớp học phần. Mỗi dòng có cấu trúc:

```
Ma lop hoc phan|Ma mon|Hoc ky|Nam hoc|Giang vien
```

Bảng 2.9: Cấu trúc các trường trong file sections.txt

Vị trí	Tên trường	Kiểu	Ghi chú
0	Mã lớp học phần	string	Khóa định danh lớp học phần.
1	Mã môn	string	Liên kết tới courses.txt.
2	Học kỳ	string	Ví dụ 2025.2.
3	Năm học	string	Ví dụ 2025-2026.
4	Giảng viên	string	Tên giảng viên phụ trách.

2.2.5 Định dạng file grades.txt

File data/grades.txt là file quan trọng nhất vì chứa kết quả học tập. Mỗi dòng có cấu trúc:

```
Ma diem|MSSV|Ma mon|Ma lop hoc phan|QT|GK|CK|Tong10
```

Bảng 2.10: Cấu trúc các trường trong file grades.txt

Vị trí	Tên trường	Kiểu	Ghi chú
0	Mã điểm	string	Sinh theo dạng G_MSSV_MAMON.
1	MSSV	string	Liên kết tới sinh viên.
2	Mã môn	string	Liên kết tới môn học.
3	Mã lớp học phần	string	Liên kết tới lớp học phần.
4	Điểm quá trình	double	Phải thuộc khoảng 0 đến 10.
5	Điểm giữa kỳ	double	Phải thuộc khoảng 0 đến 10.
6	Điểm cuối kỳ	double	Phải thuộc khoảng 0 đến 10.
7	Điểm tổng	double	Được tính tự động theo công thức trọng số.

2.2.6 Tổng hợp các file dữ liệu

Bảng 2.11: Tổng hợp các file dữ liệu của hệ thống

Đường dẫn	Danh sách trong RAM	Vai trò
data/students.txt	LinkedList<Student>	Lưu danh sách sinh viên và thông tin cá nhân.
data/courses.txt	LinkedList<Course>	Lưu danh mục môn học và số tín chỉ.
data/sections.txt	LinkedList<ClassSection>	Lưu thông tin lớp học phần.
data/grades.txt	LinkedList<Grade>	Lưu điểm thành phần và điểm tổng kết.
gui/dashboard.html	Dữ liệu xuất từ các danh sách	Giao diện Dashboard HTML, dùng để minh họa và trình bày trực quan.

2.2.7 Cơ chế đọc file

Khi chương trình khởi động, hàm `loadAll` gọi bốn hàm đọc file tương ứng. Mỗi hàm mở file, đọc từng dòng, tách chuỗi theo ký tự `|`, kiểm tra số lượng trường và tạo đối tượng dữ liệu. Sau đó đối tượng được thêm vào cuối danh sách liên kết bằng `addBack`.

Thuật toán đọc dữ liệu

1. Mở file cần đọc bằng `ifstream`.
2. Nếu không mở được file, trả về danh sách rỗng và không làm chương trình dừng.
3. Đọc từng dòng, bỏ qua dòng rỗng.
4. Tách dòng thành mảng chuỗi bằng hàm `splitPipe`.
5. Nếu số trường hợp lệ, khởi tạo đối tượng tương ứng.

6. Gọi `addBack` để đưa đối tượng vào danh sách liên kết.

2.2.8 Cơ chế ghi file

Khi người dùng chọn kết thúc, chương trình gọi hàm `saveAll`. Mỗi hàm ghi file mở file ở chế độ ghi đè, duyệt danh sách liên kết từ `head` đến `NULL`, định dạng các thuộc tính thành một dòng văn bản và ghi xuống file. Trước khi ghi, hàm `safeField` thay ký tự `|` bằng ký tự khác để tránh phá cấu trúc file.

Thuật toán ghi dữ liệu

1. Bảo đảm thư mục `data` tồn tại.
2. Mở file bằng `ofstream` ở chế độ ghi.
3. Duyệt từng node trong danh sách liên kết.
4. Lấy đối tượng dữ liệu trong node hiện tại.
5. Ghép các trường bằng dấu `|` theo đúng định dạng.
6. Ghi dòng dữ liệu xuống file và chuyển sang node kế tiếp.
7. Đóng file và trả về trạng thái thành công hoặc thất bại.

CHƯƠNG 3.

CẤU TRÚC CHƯƠNG TRÌNH VÀ CÁC MODULE CHỨC NĂNG

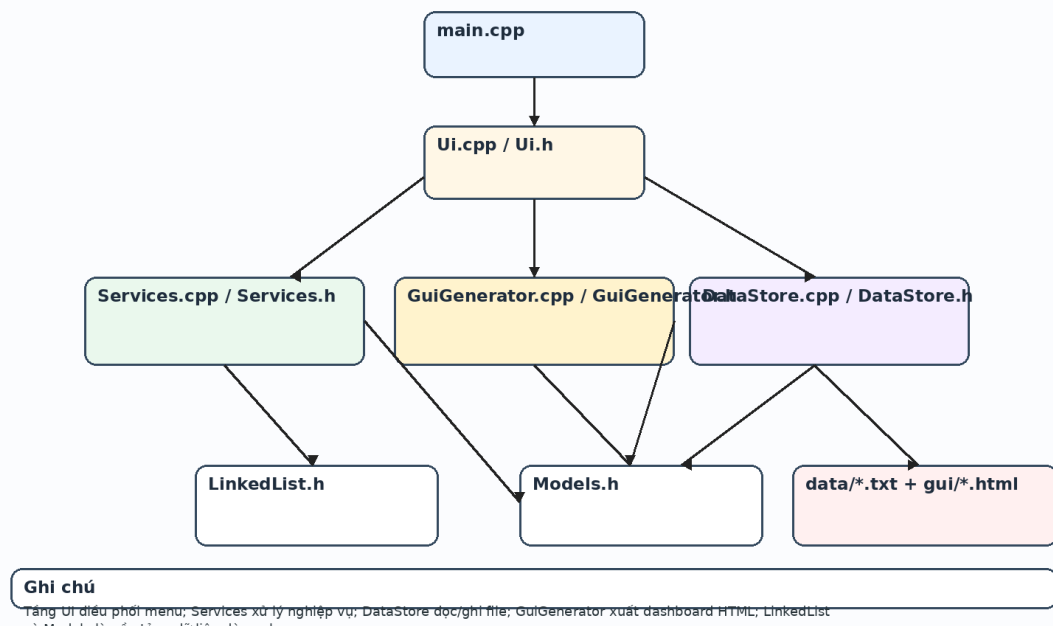
3.1 Sơ đồ tổ chức cấu trúc chương trình

Chương trình được tổ chức theo hướng module hóa. Mỗi module đảm nhiệm một nhóm trách nhiệm riêng, giúp mã nguồn dễ đọc và dễ kiểm tra. Cách tổ chức này cũng giúp báo cáo có thể mô tả từng phần rõ ràng: dữ liệu, xử lý nghiệp vụ, đọc ghi file, giao diện và kiểm thử.

Bảng 3.1: Danh sách các module và vai trò trong hệ thống

Module	Loại	Vai trò chính
main.cpp	Khởi chạy	Gọi hàm <code>runProgram</code> để bắt đầu chương trình.
Models.h	Dữ liệu	Định nghĩa <code>Student</code> , <code>Course</code> , <code>ClassSection</code> , <code>Grade</code> .
LinkedList.h	Cấu trúc dữ liệu	Cài đặt <code>Node</code> và <code>LinkedList</code> bằng con trỏ.
DataStore.h/cpp	File I/O	Đọc và ghi các file text trong thư mục <code>data</code> .
Services.h/cpp	Nghiệp vụ	Tìm kiếm, tính điểm, quy đổi GPA, xếp loại, kiểm tra dữ liệu, sắp xếp.
Ui.h/cpp	Giao diện CLI	In menu, nhận lựa chọn, gọi các hàm nghiệp vụ và báo cáo.
GuiGenerator.h/cpp	Giao diện GUI	Xuất file <code>gui/dashboard.html</code> để hiển thị Dashboard bằng trình duyệt.
tests/test_runner.cpp	Kiểm thử	Kiểm thử tự động các hàm tính toán và thao tác danh sách.

Sơ đồ phụ thuộc giữa các module C++



Hình 3.1: Sơ đồ phụ thuộc giữa các module C++

3.2 Mô tả chi tiết các module

3.2.1 Module main.cpp

`main.cpp` là điểm vào của chương trình. File này được giữ rất ngắn để thể hiện đúng nguyên tắc: điểm khởi chạy chỉ nên gọi bộ điều khiển chính, không nên chứa logic nghiệp vụ.

```

1 #include "../include/Ui.h"
2
3 int main() {
4     runProgram();
5     return 0;
6 }
  
```

Listing 3.1: Hàm main của chương trình

Cách viết này giúp nếu sau này muốn thay đổi giao diện, ví dụ chuyển từ CLI sang giao diện khác, nhóm chỉ cần thay đổi hàm điều phối mà không ảnh hưởng đến các module dữ liệu.

3.2.2 Module Models.h

Module `Models.h` chứa các kiểu dữ liệu thuần. Các struct không có phương thức phức tạp, nhờ đó việc đọc ghi file và truyền dữ liệu giữa các module đơn giản hơn. Các kiểu dữ liệu đều có trường `id` vì lớp `LinkedList` sử dụng trường này trong hàm `findById` và `removeById`.

```

1 struct Student {
  
```



```
2   string id;
3   string fullName;
4   string className;
5   string birthDate;
6   string gender;
7   string phone;
8   string email;
9 };
10
11 struct Grade {
12     string id;
13     string studentId;
14     string courseId;
15     string sectionId;
16     double processScore;
17     double midtermScore;
18     double finalScore;
19     double total10;
20 };
```

Listing 3.2: Các kiểu dữ liệu chính trong Models.h

3.2.3 Module LinkedList.h

LinkedList.h là module thể hiện rõ nhất yêu cầu tự cài đặt cấu trúc dữ liệu. Module này quản lý con trỏ head, cấp phát node bằng new và giải phóng bằng delete. Các thao tác thêm, tìm, xóa và sắp xếp đều duyệt trực tiếp trên các node.

```
1 void addBack(const T& value) {
2     Node<T>* node = new Node<T>(value);
3     if (head == 0) {
4         head = node;
5         return;
6     }
7     Node<T>* cur = head;
8     while (cur->next != 0) cur = cur->next;
9     cur->next = node;
10 }
```

Listing 3.3: Phương thức thêm node vào cuối danh sách

```
1 bool removeById(const string& id) {
2     if (head == 0) return false;
3     if (head->data.id == id) {
4         Node<T>* temp = head;
5         head = head->next;
6         delete temp;
```

```

7     return true;
8 }
9 Node<T>* prev = head;
10 Node<T>* cur = head->next;
11 while (cur != 0) {
12     if (cur->data.id == id) {
13         prev->next = cur->next;
14         delete cur;
15         return true;
16     }
17     prev = cur;
18     cur = cur->next;
19 }
20 return false;
21 }

```

Listing 3.4: Phương thức xóa node theo mã định danh

3.2.4 Module DataStore.h/cpp

Module DataStore đảm nhiệm toàn bộ đọc ghi file. Hàm `splitPipe` được tự viết để tách chuỗi theo dấu `|`. Nhóm không dùng thư viện phân tích JSON, XML hay các cấu trúc nâng cao vì file dữ liệu được thiết kế đơn giản.

```

1 int splitPipe(const string& line, string parts[], int maxParts) {
2     int count = 0;
3     string current = "";
4     for (int i = 0; i < (int)line.length(); i++) {
5         if (line[i] == '|') {
6             if (count < maxParts) parts[count++] = current;
7             current = "";
8         } else {
9             current += line[i];
10        }
11    }
12    if (count < maxParts) parts[count++] = current;
13    return count;
14 }

```

Listing 3.5: Hàm tách dòng dữ liệu theo dấu gạch đứng

3.2.5 Module Services.h/cpp

Module Services chứa các hàm nghiệp vụ. Các hàm quan trọng gồm tính điểm tổng, quy đổi hệ 4, xếp loại, tìm kiếm sinh viên, tìm kiếm môn, tính điểm trung bình có trọng số tín chỉ, kiểm tra điểm và thêm hoặc cập nhật điểm.

Bảng 3.2: Các hàm nghiệp vụ chính trong Services.cpp

Hàm	Vai trò
calculateTotal10	Tính điểm tổng hệ 10 theo công thức 20% quá trình, 30% giữa kỳ, 50% cuối kỳ.
convertToScale4	Quy đổi điểm hệ 10 sang điểm hệ 4 theo các ngưỡng xếp loại.
classifyByAverage10	Trả về xếp loại học lực: Xuất sắc, Giỏi, Khá, Trung bình, Yếu, Kém.
calculateAverage10	Tính điểm trung bình hệ 10 có trọng số tín chỉ.
calculateGPA4	Tính GPA hệ 4 có trọng số tín chỉ.
addOrUpdateGrade	Kiểm tra dữ liệu và thêm mới hoặc cập nhật điểm.
sortStudentsByName	Sắp xếp sinh viên theo họ tên bằng nổi bọt.
validateDate	Kiểm tra định dạng ngày sinh YYYY-MM-DD.
validateScore	Kiểm tra điểm thuộc khoảng 0 đến 10.

```

1 double calculateTotal10(double process, double midterm, double finalScore)
2 {
3     return round2(process * 0.2 + midterm * 0.3 + finalScore * 0.5);
4 }

```

Listing 3.6: Công thức tính điểm tổng hệ 10

Công thức được đặt trong hàm riêng để bảo đảm tính nhất quán. Khi nhập điểm mới, cập nhật điểm hoặc chạy cập nhật toàn bộ điểm tổng, chương trình đều gọi cùng một hàm này. Điều đó tránh trường hợp mỗi nơi tính một công thức khác nhau.

3.2.6 Module GuiGenerator.h/cpp

Module `GuiGenerator` được bổ sung để đáp ứng yêu cầu có giao diện trực quan. Thay vì phụ thuộc vào Qt, GTK hoặc thư viện GUI nặng, nhóm chọn cách xuất giao diện HTML tĩnh. Đây là giải pháp nhẹ, dễ chạy trên nhiều máy và vẫn được sinh trực tiếp từ dữ liệu của chương trình C++.

```

1 bool exportGuiDashboard(LinkedList<Student>& students,
2                          LinkedList<Course>& courses,
3                          LinkedList<ClassSection>& sections,
4                          LinkedList<Grade>& grades,
5                          const string& outputPath);

```

Listing 3.7: Khai báo hàm xuất Dashboard GUI

Hàm `exportGuiDashboard` duyệt các danh sách liên kết để tính số lượng sinh viên, môn học, lớp học phần, bản ghi điểm và điểm trung bình chung. Sau đó hàm ghi ra file `gui/dashboard.html` gồm các thẻ thống kê, bảng sinh viên, bảng điểm lớp MI3310-01 và các trạng thái kiểm soát nghiệp vụ. Người dùng chỉ cần mở file này bằng trình duyệt để xem giao diện.

3.2.7 Module Ui.h/cpp

Module Ui phụ trách giao diện dòng lệnh. Module này in menu, nhận lựa chọn từ người dùng và gọi các hàm tương ứng. Đây là nơi kết nối người dùng với các module còn lại. Giao diện được thiết kế dạng bảng với độ rộng cố định, dùng setw để căn cột nhằm tránh dữ liệu bị dính hoặc khó đọc.

```

1 void printMainMenu() {
2     cout << " 1. Hien thi danh sach sinh vien\n";
3     cout << " 2. Them sinh vien\n";
4     cout << " 6. Nhap / cap nhat diem theo MSSV va ma mon\n";
5     cout << " 10. Bao cao in de nop va ky danh sach thi\n";
6     cout << " 11. Xuat giao dien GUI Dashboard dang HTML\n";
7     cout << " 0. Luu du lieu va ket thuc\n";
8 }

```

Listing 3.8: Một phần menu chính của chương trình

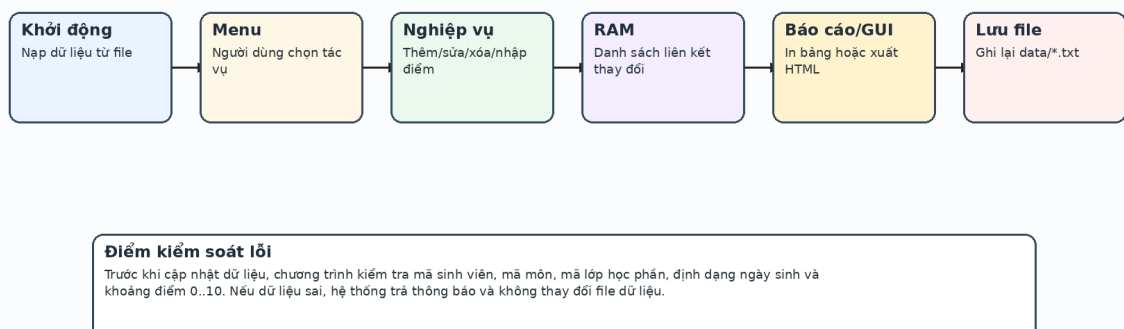
3.2.8 Module kiểm thử test_runner.cpp

File test_runner.cpp kiểm tra các hàm quan trọng mà không cần nhập tay qua menu. Cách kiểm thử này giúp phát hiện lỗi sớm khi thay đổi code. Các ca kiểm thử bao gồm tính điểm tổng, quy đổi hệ 4, xếp loại, kiểm tra ngày sinh, thêm node, tìm kiếm, sắp xếp, xóa node, thêm điểm hợp lệ, tính trung bình có trọng số và chặn điểm ngoài khoảng.

3.3 Quy trình vận hành hệ thống

Luồng xử lý tổng thể được mô tả trong Hình 3.2. Vòng lặp chính chỉ kết thúc khi người dùng chọn 0. Mỗi chức năng được thực hiện độc lập, sau đó chương trình quay lại menu.

Luồng xử lý tổng thể của chương trình



Hình 3.2: Luồng xử lý tổng thể của chương trình

3.3.1 Luồng khởi động

Khi chạy chương trình, hàm `runProgram` tạo bốn danh sách liên kết rỗng. Sau đó `loadAll` nạp dữ liệu từ file text. Tiếp theo, hàm `updateAllTotalScores` tính lại điểm tổng của các bản ghi điểm để bảo đảm dữ liệu nhất quán ngay cả khi file đã được sửa thủ công.

3.3.2 Luồng xử lý theo menu

Bảng 3.3: Workflow chi tiết theo từng lựa chọn menu

Chọn	Tác vụ	Luồng xử lý
1	Hiển thị sinh viên	Duyệt danh sách sinh viên, với mỗi sinh viên tính số tín chỉ, điểm TB10, GPA4 và xếp loại rồi in thành bảng.
2	Thêm sinh viên	Nhập thông tin, kiểm tra trùng MSSV, kiểm tra ngày sinh, thêm node mới vào danh sách.
3	Cập nhật sinh viên	Tìm sinh viên theo MSSV, nếu tồn tại thì cập nhật các trường người dùng nhập.
4	Xóa sinh viên	Gọi <code>removeById</code> để loại node sinh viên khỏi danh sách.
5	Hiển thị môn học	Duyệt danh sách môn học và in mã môn, tên môn, số tín chỉ.
6	Nhập/cập nhật điểm	Kiểm tra sinh viên, môn học, lớp học phần và khoảng điểm; sau đó thêm hoặc cập nhật bản ghi điểm.
7	Bảng điểm cá nhân	Tìm sinh viên, duyệt danh sách điểm theo MSSV và in bảng điểm kèm GPA.
8	Bảng điểm lớp học phần	Tìm lớp học phần, duyệt danh sách điểm theo mã lớp và in bảng tổng hợp.
9	Sắp xếp	Chọn tiêu chí MSSV hoặc họ tên, gọi bubble sort và in lại danh sách.
10	Danh sách thi	Tìm lớp học phần, in báo cáo có cột ký tên và trạng thái đủ điều kiện.
11	Xuất GUI	Gọi <code>exportGuiDashboard</code> để tạo file <code>gui/dashboard.html</code> .

Chọn	Tác vụ	Luồng xử lý
0	Kết thúc	Gọi <code>saveAll</code> để ghi dữ liệu xuống file và thoát chương trình.

3.4 Các thuật toán chính

3.4.1 Tìm kiếm tuyến tính

Tìm kiếm tuyến tính được dùng trong hầu hết nghiệp vụ vì dữ liệu lưu bằng danh sách liên kết. Thuật toán bắt đầu từ `head`, so sánh mã định danh của từng node với khóa cần tìm, nếu khớp thì trả về con trỏ dữ liệu, nếu duyệt đến cuối mà không thấy thì trả về `NULL`.

Độ phức tạp: $O(n)$, trong đó n là số node của danh sách. Với dữ liệu lớp học phần, n thường không lớn nên thuật toán phù hợp. Ngoài ra, việc tự cài đặt tuyến tính đáp ứng yêu cầu không dùng hàm tìm kiếm dựng sẵn.

3.4.2 Sắp xếp nổi bọt

Sắp xếp nổi bọt được chọn vì dễ cài đặt trên danh sách liên kết và dễ giải thích trong báo cáo. Nhóm không dùng `std::sort`. Hàm `bubbleSort` nhận con trỏ hàm so sánh để có thể sắp xếp nhiều kiểu dữ liệu theo nhiều tiêu chí.

```

1 void bubbleSort(int (*compare)(const T&, const T&)) {
2     if (head == 0 || head->next == 0) return;
3     bool swapped;
4     do {
5         swapped = false;
6         Node<T>* cur = head;
7         while (cur->next != 0) {
8             if (compare(cur->data, cur->next->data) > 0) {
9                 T tmp = cur->data;
10                cur->data = cur->next->data;
11                cur->next->data = tmp;
12                swapped = true;
13            }
14            cur = cur->next;
15        }
16    } while (swapped);
17 }
```

Listing 3.9: Thuật toán sắp xếp nổi bọt trên LinkedList

3.4.3 Tính GPA có trọng số tín chỉ

Điểm trung bình không được tính bằng trung bình cộng đơn giản, vì mỗi môn có số tín chỉ khác nhau. Công thức nhóm sử dụng:

$$TB10 = \frac{\sum_{i=1}^n DiemTong_i \times TinChi_i}{\sum_{i=1}^n TinChi_i}$$

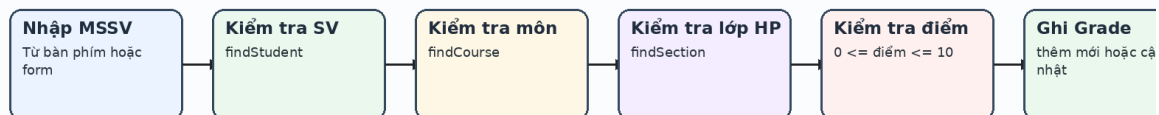
$$GPA4 = \frac{\sum_{i=1}^n QuyDoi4(DiemTong_i) \times TinChi_i}{\sum_{i=1}^n TinChi_i}$$

Trong đó $DiemTong_i$ là điểm tổng hệ 10 của môn thứ i . Hàm tính GPA duyệt toàn bộ danh sách điểm, chỉ lấy các bản ghi thuộc sinh viên cần tính, sau đó tra cứu số tín chỉ trong danh sách môn học.

3.4.4 Kiểm tra dữ liệu nhập điểm

Nhập điểm là thao tác có rủi ro cao vì liên quan đến nhiều khóa dữ liệu. Do đó, hàm `addOrUpdateGrade` kiểm tra tuần tự: sinh viên có tồn tại không, môn học có tồn tại không, lớp học phần có tồn tại không, điểm có nằm trong khoảng 0 đến 10 không. Nếu một điều kiện sai, hàm trả về `false` và thông báo lỗi. Dữ liệu trong danh sách liên kết và file `grades.txt` không bị thay đổi.

Quy trình kiểm tra dữ liệu khi nhập điểm



Bẫy logic

Nếu một bước kiểm tra thất bại, hàm `addOrUpdateGrade` trả `false` và thông báo lỗi. Dữ liệu trong danh sách liên kết và file `grades.txt` không bị thay đổi.

Hình 3.3: Quy trình kiểm tra dữ liệu khi nhập điểm

3.5 Kỹ thuật đặt tên và trình bày mã nguồn

Mã nguồn được đặt tên theo hướng dễ hiểu, ưu tiên mô tả vai trò. Tên hàm dùng động từ hoặc cụm động từ như `loadStudents`, `saveGrades`, `calculateGPA4`, `printTranscriptOfStudent`. Tên biến khóa dùng hậu tố `Id` như `studentId`,

courseId, sectionId. Tên file chia theo module để người đọc nhanh chóng biết nơi cần sửa khi có lỗi.

Bảng 3.4: Quy ước đặt tên trong mã nguồn

Thành phần	Quy ước	Ví dụ
Kiểu dữ liệu	PascalCase	Student, ClassSection, LinkedList
Hàm xử lý	camelCase, bắt đầu bằng động từ	calculateAverage10, printStudents
Biến cục bộ	camelCase ngắn gọn	studentId, avg10, cur
File header	Trùng tên module và đuôi .h	Services.h, DataStore.h
File nguồn	Trùng tên module và đuôi .cpp	Services.cpp, Ui.cpp
File dữ liệu	Tên số nhiều, chữ thường	students.txt, grades.txt

CHƯƠNG 4.

THIẾT KẾ VÀ HIỆN THỰC HÓA GIAO DIỆN NGƯỜI DÙNG

4.1 Mục tiêu thiết kế giao diện

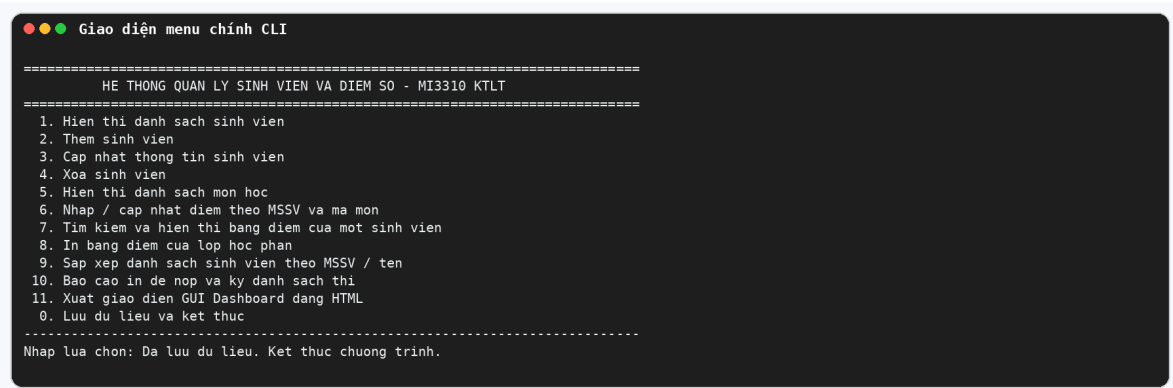
Giao diện của chương trình được thiết kế với ba mục tiêu chính: dễ dùng, dễ kiểm tra và phù hợp với môi trường nộp bài. Vì chương trình chính viết bằng C++ và cần chạy ổn định trên nhiều máy, nhóm giữ giao diện dòng lệnh làm giao diện tương tác chính. Bên cạnh đó, nhóm bổ sung giao diện Dashboard HTML để trình bày dữ liệu đẹp hơn, trực quan hơn trong báo cáo.

Các yêu cầu giao diện được xác định như sau:

- Menu cần rõ ràng, đánh số chức năng từ 0 đến 11.
- Bảng in ra console phải có cột căn chỉnh, không dính chữ, không mất dữ liệu quan trọng.
- Các thông báo lỗi phải ngắn gọn và nêu đúng nguyên nhân.
- Giao diện Dashboard phải có bố cục hiện đại: sidebar, thẻ thống kê, bảng dữ liệu và trạng thái nghiệp vụ.
- Dữ liệu trên Dashboard phải được sinh từ chương trình, không nhập tay rời rạc.

4.2 Giao diện dòng lệnh CLI

Giao diện CLI là nơi người dùng thao tác trực tiếp. Hình 4.1 thể hiện menu chính sau khi chạy chương trình. Menu được chia thành nhóm quản lý, điểm số, báo cáo và kết thúc.



```

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hiển thị danh sách sinh viên
2. Thêm sinh viên
3. Cập nhật thông tin sinh viên
4. Xóa sinh viên
5. Hiển thị danh sách môn học
6. Nhập / cập nhật điểm theo MSSV và mã môn
7. Tìm kiếm và hiển thị bảng điểm của một sinh viên
8. In bảng điểm của lớp học phân
9. Sắp xếp danh sách sinh viên theo MSSV / tên
10. Báo cáo in đề nộp và ký danh sách thi
11. Xuất giao diện GUI Dashboard dạng HTML
0. Lưu dữ liệu và kết thúc
=====
Nhập lựa chọn: Đã lưu dữ liệu. Kết thúc chương trình.

```

Hình 4.1: Giao diện menu chính của chương trình CLI

4.2.1 Nguyên tắc trình bày bảng trong CLI

Các bảng in ra console sử dụng `setw` để cố định độ rộng cột. Với các chuỗi dài như họ tên hoặc tên môn học, hàm `cutStr` rút gọn và thêm dấu ba chấm. Điều này giúp bảng không bị lệch cột khi dữ liệu dài. Trong báo cáo, các ảnh chụp được dựng với font đơn cách để thể hiện rõ bố cục bảng.

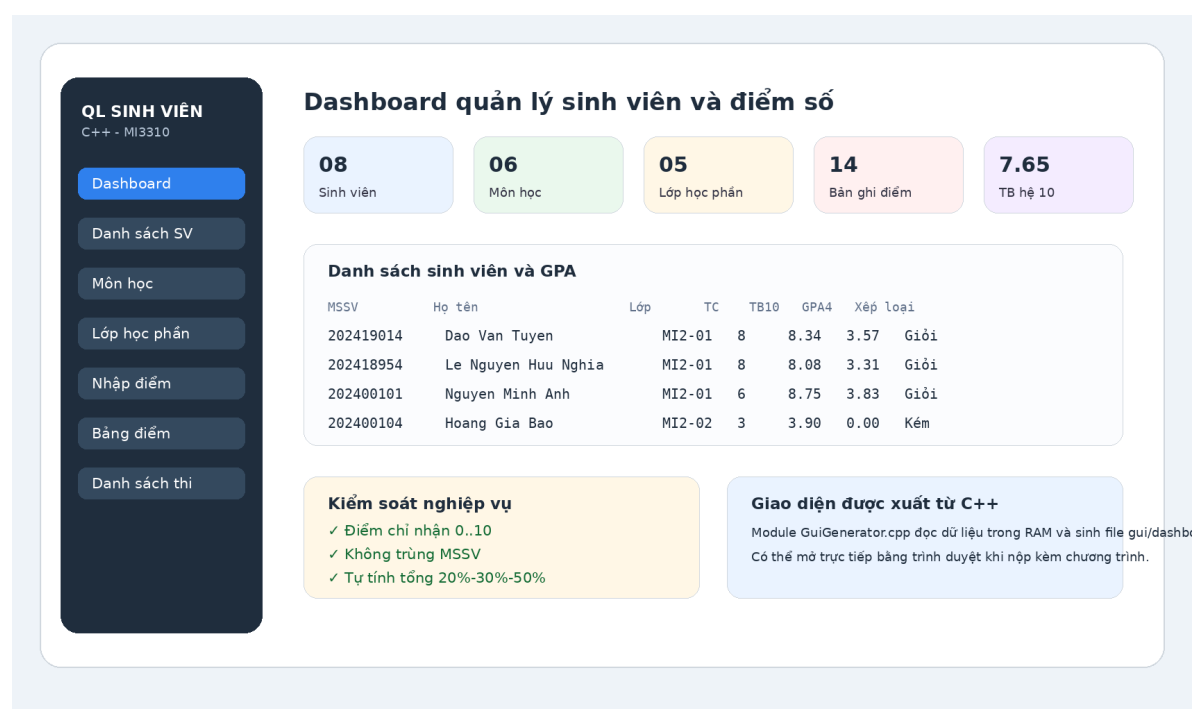
Bảng 4.1: Nguyên tắc thiết kế bảng CLI

Vấn đề	Giải pháp	Lợi ích
Tên sinh viên dài	Dùng <code>cutStr</code> để cắt ở độ dài phù hợp.	Không làm tràn bảng, vẫn nhận biết được họ tên.
Số thập phân không đều	Dùng <code>fixed</code> và <code>setprecision</code> .	Điểm hiển thị thống nhất.
Các dòng khó phân biệt	In đường kẻ bằng ký tự <code>-</code> hoặc <code>=</code> .	Bảng dễ đọc hơn khi chạy trong terminal.
Nhiều cột dữ liệu	Chỉ hiển thị các cột quan trọng trong từng màn hình.	Tránh quá tải thông tin.

4.3 Giao diện Dashboard HTML

4.3.1 Lý do chọn Dashboard HTML

Để xây dựng GUI bằng C++ có thể dùng các thư viện như Qt hoặc GTK, nhưng các thư viện này làm tăng độ phức tạp cài đặt và có thể gây khó khăn khi nộp bài trên máy khác. Vì vậy, nhóm lựa chọn giải pháp xuất giao diện HTML. C++ vẫn là ngôn ngữ xử lý chính; module `GuiGenerator.cpp` chỉ ghi ra một file HTML có CSS. Người dùng mở file này bằng trình duyệt để xem giao diện. Cách này nhẹ, dễ chạy, không cần cài đặt thêm thư viện GUI.



Hình 4.2: Giao diện Dashboard HTML của hệ thống

4.3.2 Bố cục tổng thể Dashboard

Dashboard gồm hai vùng chính. Vùng bên trái là sidebar chứa tên ứng dụng và các mục chức năng. Vùng bên phải là nội dung chính gồm tiêu đề, thẻ thống kê, bảng danh sách sinh viên, bảng điểm lớp học phần và khối kiểm soát nghiệp vụ. Màu nền được chọn nhẹ, các khối bo góc để tạo cảm giác hiện đại nhưng vẫn rõ ràng.

Bảng 4.2: Cấu trúc bố cục Dashboard HTML

Khu vực	Thành phần	Vai trò
Sidebar	Logo, mô tả môn học, danh sách mục	Tạo cảm giác giống ứng dụng quản trị.
Top bar	Tiêu đề và nhãn dữ liệu từ file text	Giúp người xem hiểu nguồn dữ liệu.
Stat cards	Số sinh viên, số môn, số lớp học phần, bản ghi điểm, điểm TB	Tóm tắt nhanh trạng thái hệ thống.
Bảng sinh viên	MSSV, họ tên, lớp, tín chỉ, TB10, GPA4, xếp loại	Hiển thị dữ liệu cốt lõi.
Bảng điểm lớp	Điểm của lớp MI3310-01	Minh họa chức năng báo cáo lớp học phần.
Khối nghiệp vụ	Các ràng buộc đang bật	Thể hiện kiểm soát logic chương trình.

4.3.3 Giao diện form nhập điểm

Trong chương trình chính, nhập điểm được thực hiện qua CLI. Tuy nhiên, để trình bày trực quan luồng nhập điểm, nhóm thiết kế mô phỏng form GUI trong báo cáo. Form gồm các trường MSSV, mã môn học, mã lớp học phần, điểm quá trình, điểm giữa kỳ và điểm cuối kỳ. Khi nhấn lưu, chương trình cần kiểm tra dữ liệu rồi tự tính điểm tổng.

Form nhập / cập nhật điểm

MSSV	Điểm quá trình
202419014	8.5
Mã môn học	Điểm giữa kỳ
MI3310	8.0
Mã lớp học phần	Điểm cuối kỳ
MI3310-01	9.0

Lưu điểm **Điểm tổng tự tính: 8.60** **Chặn điểm ngoài 0..10**

Luồng xử lý: kiểm tra MSSV -> kiểm tra mã môn -> kiểm tra lớp học phần -> kiểm tra điểm -> thêm/cập nhật Grade -> ghi file.

Hình 4.3: Thiết kế giao diện form nhập/cập nhật điểm

Điểm đáng chú ý là form không chỉ đẹp mà còn phản ánh đúng logic xử lý. Nếu nhập điểm 11 hoặc -1, hệ thống phải chặn và không ghi vào danh sách. Nếu MSSV không tồn tại, hệ thống báo lỗi trước khi kiểm tra điểm. Việc này giúp dữ liệu luôn nhất quán.

4.3.4 Giao diện báo cáo in danh sách thi

Danh sách thi là báo cáo có tính chất nộp/in nên cần bố cục rõ ràng. Báo cáo gồm thông tin trường, khoa, tên báo cáo, môn học, lớp học phần, học kỳ, giảng viên, bảng sinh viên và vùng ký xác nhận.

Màn hình báo cáo in danh sách thi

ĐẠI HỌC BÁCH KHOA HÀ NỘI - KHOA TOÁN TIN
DANH SÁCH SINH VIÊN ĐỦ ĐIỀU KIỆN DỰ THI / KÝ TÊN
 Môn học: MI3310 - Kỹ thuật lập trình | Lớp HP: MI3310-01 | GV: TS. Vũ Thành Nam

STT	MSSV	Họ tên	Lớp	QT	Tổng	Trạng thái	Ký tên
1	202419014	Dao Van Tuyen	MI2-01	8.5	8.60	Đủ điều kiện	
2	202418954	Le Nguyen Huu Nghia	MI2-01	8.0	8.40	Đủ điều kiện	
3	202400104	Hoang Gia Bao	MI2-02	3.5	3.90	Cần xem xét	

Hà Nội, ngày tháng năm 2026
 Cán bộ coi thi Giảng viên phụ trách Đại diện lớp

Hình 4.4: Thiết kế giao diện báo cáo in danh sách thi

4.4 Hiện thực hóa chức năng xuất GUI

Trong menu chính, người dùng chọn số 11 để xuất Dashboard. Khi đó chương trình gọi hàm `exportGuiDashboard`. Hàm này duyệt các danh sách liên kết để lấy dữ liệu, tính toán các chỉ số tổng hợp và ghi ra file HTML. Dữ liệu hiển thị trên GUI vì vậy luôn đồng nhất với dữ liệu trong chương trình.

```

1 else if (choice == 11) {
2     bool ok = exportGuiDashboard(students, courses, sections, grades,
3                                 "gui/dashboard.html");
4     cout << (ok ? "Đã xuất GUI Dashboard: gui/dashboard.html\n"
5               : "Không thể xuất GUI Dashboard.\n");
6 }

```

Listing 4.1: Gọi chức năng xuất GUI trong Ui.cpp

```

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
6. Nhap / cap nhat diem theo MSSV va ma mon
7. Tim kiem va hien thi bang diem cua mot sinh vien
8. In bang diem cua lop hoc phan
9. Sap xep danh sach sinh vien theo MSSV / ten
10. Bao cao in de nop va ky danh sach thi
11. Xuat giao dien GUI Dashboard dang HTML
0. Luu du lieu va ket thuc
=====
Nhap lua chon: Da xuat GUI Dashboard: gui/dashboard.html
=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
...
    
```

Hình 4.5: Kết quả chạy chức năng xuất Dashboard GUI

4.5 Đánh giá giao diện

Giao diện CLI và Dashboard HTML đáp ứng hai mục tiêu khác nhau. CLI phù hợp để chạy chương trình, nhập dữ liệu và kiểm thử trực tiếp. Dashboard HTML phù hợp để trình bày dữ liệu đẹp hơn trong báo cáo hoặc khi cần xem nhanh trạng thái hệ thống. Cách kết hợp này giúp sản phẩm vừa đúng yêu cầu lập trình C++ vừa có phần trình bày trực quan.

Bảng 4.3: Đánh giá ưu điểm và hạn chế của giao diện

Loại giao diện	Ưu điểm	Hạn chế
CLI	Dễ biên dịch, chạy ổn định, nhập liệu nhanh, phù hợp yêu cầu môn học.	Chưa đẹp như ứng dụng desktop, phụ thuộc độ rộng terminal.
Dashboard HTML	Bố cục trực quan, đẹp, mở được bằng trình duyệt, không cần thư viện GUI nặng.	Chủ yếu hiển thị và báo cáo, chưa nhập liệu trực tiếp trong trình duyệt.
Mô phỏng form GUI	Giúp báo cáo thể hiện rõ ý tưởng giao diện nhập điểm và báo cáo in.	Là thiết kế minh họa, phần nhập thật vẫn thực hiện qua CLI.

CHƯƠNG 5.

KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ

5.1 Môi trường biên dịch và chạy chương trình

Chương trình được biên dịch bằng g++ với chuẩn C++11. File `Makefile` hỗ trợ hai lệnh chính: `make` để biên dịch chương trình và `make test` để biên dịch, chạy bộ kiểm thử. Cách tổ chức này giúp việc nộp và kiểm tra bài thuận tiện hơn.

Lệnh biên dịch và chạy:

```
make  
./student_manager  
make test
```

5.2 Kết quả chạy các chức năng chính

5.2.1 Hiện thị danh sách sinh viên

Chức năng số 1 hiển thị danh sách sinh viên kèm điểm trung bình và GPA. Kết quả cho thấy chương trình không chỉ in dữ liệu sinh viên mà còn tự tính toán chỉ, điểm trung bình và xếp loại từ danh sách điểm.

```

Danh sách sinh viên và GPA

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
6. Nhap / cap nhat diem theo MSSV va ma mon
7. Tim kiem va hien thi bang diem cua mot sinh vien
8. In bang diem cua lop hoc phan
9. Sap xep danh sach sinh vien theo MSSV / ten
10. Bao cao in de nop va ky danh sach thi
11. Xuat giao dien GUI Dashboard dang HTML
0. Luu du lieu va ket thuc

-----
Nhap lua chon:
DANH SACH SINH VIEN
-----
MSSV      Ho ten      Lop      Ngay sinh  Gioi    TC      TB10      GPA4      Xep loai
-----
202419014  Dao Van Tuyen  MI2-01   2006-05-19 Nam      8        8.31      3.44      Gioi
202400106  Do Thanh Dat   MI2-03   2006-09-05 Nam      2        5.90      2.00      Trung binh
202400104  Hoang Gia Bao  MI2-02   2006-06-09 Nam      3        3.90      0.00      Kem
202418954  Le Nguyen Huu Nghia  MI2-01   2006-08-12 Nam      8        8.08      3.19      Gioi
202400101  Nguyen Minh Anh  MI2-01   2006-01-03 Nu       6        8.75      3.75      Gioi
202400103  Pham Thu Ha    MI2-02   2006-03-20 Nu       3        8.00      3.50      Gioi
202400102  Tran Quang Huy  MI2-01   2006-02-14 Nam      6        6.78      2.50      Kha
202400105  Vu Khanh Linh  MI2-02   2006-11-28 Nu       3        7.10      3.00      Kha
-----

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
...

```

Hình 5.1: Kết quả hiển thị danh sách sinh viên

5.2.2 Tìm kiếm bảng điểm cá nhân

Chức năng số 7 cho phép nhập MSSV và in bảng điểm của sinh viên tương ứng. Nếu MSSV tồn tại, hệ thống in thông tin sinh viên, danh sách môn, điểm thành phần, điểm tổng, hệ 4, điểm trung bình và xếp loại.

```

Bảng điểm cá nhân

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
6. Nhap / cap nhat diem theo MSSV va ma mon
7. Tim kiem va hien thi bang diem cua mot sinh vien
8. In bang diem cua lop hoc phan
9. Sap xep danh sach sinh vien theo MSSV / ten
10. Bao cao in de nop va ky danh sach thi
11. Xuat giao dien GUI Dashboard dang HTML
0. Luu du lieu va ket thuc

-----
Nhap lua chon: Nhap MSSV:
BANG DIEM CA NHAN
=====
MSSV: 202419014 | Ho ten: Dao Van Tuyen | Lop: MI2-01
-----
Ma mon    Ten mon          TC    QT    GK    CK    Tong    He4
-----
MI3310    Ky thuat lap trinh  3     8.5   8.0   9.0   8.60    3.70
MI1110    Giai tich 1       3     7.0   7.5   8.0   7.65    3.00
IT1110    Tin hoc dai cuong  2     9.0   8.5   9.0   8.85    3.70
-----
Diem trung binh he 10: 8.31 | GPA he 4: 3.44 | Xep loai: Gioi

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
...

```

Hình 5.2: Kết quả tìm kiếm và hiển thị bảng điểm cá nhân

5.2.3 Bảng điểm lớp học phần

Chức năng số 8 in bảng điểm của một lớp học phần. Kết quả này đặc biệt quan trọng vì phù hợp yêu cầu đề tài: bảng điểm của lớp học phần. Chương trình duyệt danh sách điểm, lọc theo `sectionId`, sau đó tra cứu họ tên và lớp của từng sinh viên.

```

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
6. Nhap / cap nhat diem theo MSSV va ma mon
7. Timkiem va hien thi bang diem cua mot sinh vien
8. In bang diem cua lop hoc phan
9. Sap xep danh sach sinh vien theo MSSV / ten
10. Bao cao in de nop va ky danh sach thi
11. Xuat giao dien GUI Dashboard dang HTML
0. Luu du lieu va ket thuc
=====
Nhap lua chon: Nhap ma lop hoc phan:
BANG DIEM LOP HOC PHAN
=====
Lop HP: MI3310-01 | Mon: MI3310 - Ky thuat lap trinh | GV: TS. Vu Thanh Nam
=====
STT  MSSV      Ho ten      Lop      QT      GK      CK      Tong      Ket qua
-----
1    202419014  Dao Van Tuyen  MI2-01  8.5     8.0     9.0     8.60     Dat
2    202418954  Le Nguyen Huu Nghia  MI2-01  8.0     8.5     8.5     8.40     Dat
3    202400101  Nguyen Minh Anh  MI2-01  9.0     9.0     9.5     9.25     Dat
4    202400102  Tran Quang Huy   MI2-01  6.5     7.0     7.0     6.90     Dat
5    202400104  Hoang Gia Bao    MI2-02  3.5     4.0     4.0     3.90     Khong dat
6    202400105  Vu Khanh Linh    MI2-02  7.0     6.5     7.5     7.10     Dat
=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
6. Nhap / cap nhat diem theo MSSV va ma mon
7. Timkiem va hien thi bang diem cua mot sinh vien
...

```

Hình 5.3: Kết quả in bảng điểm lớp học phần

5.2.4 Nhập và cập nhật điểm

Chức năng số 6 cho phép nhập điểm theo MSSV, mã môn và mã lớp học phần. Trước khi ghi điểm, chương trình kiểm tra sự tồn tại của sinh viên, môn học, lớp học phần và kiểm tra khoảng điểm. Nếu bản ghi điểm đã tồn tại, chương trình cập nhật thay vì thêm bản ghi trùng.

```

    Nhập/Cập nhật điểm có kiểm tra dữ liệu

    =====
    HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
    =====
    1. Hien thi danh sach sinh vien
    2. Them sinh vien
    3. Cap nhat thong tin sinh vien
    4. Xoa sinh vien
    5. Hien thi danh sach mon hoc
    6. Nhap / cap nhat diem theo MSSV va ma mon
    7. Timkiem va hien thi bang diem cua mot sinh vien
    8. In bang diem cua lop hoc phan
    9. Sap xep danh sach sinh vien theo MSSV / ten
    10. Bao cao in de nop va ky danh sach thi
    11. Xuat giao dien GUI Dashboard dang HTML
    0. Luu du lieu va ket thuc

    -----
    Nhap lua chon: Nhap MSSV: Nhap ma mon: Nhap ma lop hoc phan: Nhap diem qua trinh, giao ky, cuoi ky: Cap nhat diem thanh cong.
    Diem tong he 10 = 8.6

    =====
    HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
    =====
    1. Hien thi danh sach sinh vien
    2. Them sinh vien
    3. Cap nhat thong tin sinh vien
    4. Xoa sinh vien
    5. Hien thi danh sach mon hoc
    6. Nhap / cap nhat diem theo MSSV va ma mon
    7. Timkiem va hien thi bang diem cua mot sinh vien
    8. In bang diem cua lop hoc phan
    9. Sap xep danh sach sinh vien theo MSSV / ten
    ...
  
```

Hình 5.4: Kết quả nhập/cập nhật điểm hợp lệ

5.2.5 Sắp xếp danh sách sinh viên

Chức năng số 9 cho phép sắp xếp sinh viên theo MSSV hoặc họ tên. Thuật toán sử dụng là sắp xếp nổi bọt tự cài đặt trên danh sách liên kết. Kết quả sau khi sắp xếp theo tên được thể hiện trong hình dưới.

```

    Sắp xếp danh sách theo tên

    =====
    HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
    =====
    1. Hien thi danh sach sinh vien
    2. Them sinh vien
    3. Cap nhat thong tin sinh vien
    4. Xoa sinh vien
    5. Hien thi danh sach mon hoc
    6. Nhap / cap nhat diem theo MSSV va ma mon
    7. Timkiem va hien thi bang diem cua mot sinh vien
    8. In bang diem cua lop hoc phan
    9. Sap xep danh sach sinh vien theo MSSV / ten
    10. Bao cao in de nop va ky danh sach thi
    11. Xuat giao dien GUI Dashboard dang HTML
    0. Luu du lieu va ket thuc

    -----
    Nhap lua chon: 1. Sap xep theo MSSV | 2. Sap xep theo ten:
    DANH SACH SINH VIEN
    -----
    MSSV      Ho ten      Lop      Ngay sinh  Gioi  TC      TB10      GPA4      Xep loai
    -----
    202419014  Dao Van Tuyen  MI2-01   2006-05-19  Nam   8        8.31      3.44      Gioi
    202400106  Do Thanh Dat   MI2-03   2006-09-05  Nam   2        5.90      2.00      Trung binh
    202400104  Hoang Gia Bao  MI2-02   2006-06-09  Nam   3        3.90      0.00      Kem
    202418954  Le Nguyen Huu Nghia  MI2-01   2006-08-12  Nam   8        8.08      3.19      Gioi
    202400101  Nguyen Minh Anh  MI2-01   2006-01-03  Nu    6        8.75      3.75      Gioi
    202400103  Pham Thu Ha    MI2-02   2006-03-20  Nu    3        8.00      3.50      Gioi
    202400102  Tran Quang Huy  MI2-01   2006-02-14  Nam   6        6.78      2.50      Kha
    202400105  Vu Khanh Linh  MI2-02   2006-11-28  Nu    3        7.10      3.00      Kha

    -----
    HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
    =====
    1. Hien thi danh sach sinh vien
    2. Them sinh vien
    3. Cap nhat thong tin sinh vien
    ...
  
```

Hình 5.5: Kết quả sắp xếp danh sách sinh viên theo tên

5.2.6 Báo cáo in danh sách thi

Chức năng số 10 tạo báo cáo phục vụ in và ký danh sách thi. Báo cáo có phần thông tin trường, khoa, môn học, lớp học phần, học kỳ, giảng viên, bảng sinh viên và cột ký tên. Đây là chức năng mở rộng hợp lý so với yêu cầu đề tài.

```

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
6. Nhap / cap nhat diem theo MSSV va ma mon
7. Tim kiem va hien thi bang diem cua mot sinh vien
8. In bang diem cua lop hoc phan
9. Sap xep danh sach sinh vien theo MSSV / ten
10. Bao cao in de nop va ky danh sach thi
11. Xuat giao dien GUI Dashboard dang HTML
0. Luu du lieu va ket thuc
=====
Nhap lua chon: Nhap ma lop hoc phan:
TRUONG DAI HOC BACH KHOA HA NOI
KHOA TOAN - TIN
=====
DANH SACH SINH VIEN DU DIEU KIEN DU THI / KY TEN
Mon hoc: MI3310 - Ky thuat lap trinh | Lop HP: MI3310-01
Hoc ky: 2025.2 | Nam hoc: 2025-2026 | Giang vien: TS. Vu Thanh Nam
=====
STT    MSSV    Ho ten    Lop    Diem QT    Diem tong    Trang thai    Ky ten
-----
1      202419014    Dao Van Tuyen    MI2-01    8.5        8.60        Du dieu kien    .....
2      202418954    Le Nguyen Huu Nghia    MI2-01    8.0        8.40        Du dieu kien    .....
3      202400101    Nguyen Minh Anh    MI2-01    9.0        9.25        Du dieu kien    .....
4      202400102    Tran Quang Huy    MI2-01    6.5        6.90        Du dieu kien    .....
5      202400104    Hoang Gia Bao    MI2-02    3.5        3.90        Can xem xet    .....
6      202400105    Vu Khanh Linh    MI2-02    7.0        7.10        Du dieu kien    .....
=====
Ha Noi, ngay .... thang .... nam 2026
Can bo coi thi    Giang vien phu trach    Dai dien lop
=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
...
  
```

Hình 5.6: Báo cáo in để nộp và ký danh sách thi

5.2.7 Xuất giao diện Dashboard GUI

Chức năng số 11 sinh file `gui/dashboard.html`. Kết quả chạy cho thấy chương trình xuất thành công file HTML, người dùng có thể mở file này bằng trình duyệt.

```

=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
6. Nhap / cap nhat diem theo MSSV va ma mon
7. Tim kiem va hien thi bang diem cua mot sinh vien
8. In bang diem cua lop hoc phan
9. Sap xep danh sach sinh vien theo MSSV / ten
10. Bao cao in de nop va ky danh sach thi
11. Xuat giao dien GUI Dashboard dang HTML
0. Luu du lieu va ket thuc
=====
Nhap lua chon: Da xuat GUI Dashboard: gui/dashboard.html
=====
HE THONG QUAN LY SINH VIEN VA DIEM SO - MI3310 KTLT
=====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Cap nhat thong tin sinh vien
4. Xoa sinh vien
5. Hien thi danh sach mon hoc
...
    
```

Hình 5.7: Kết quả xuất giao diện Dashboard HTML

5.3 Kiểm thử tự động

5.3.1 Mục tiêu kiểm thử

Kiểm thử nhằm xác nhận các hàm cốt lõi hoạt động đúng trước khi tích hợp vào giao diện. Nhóm tập trung kiểm thử những phần có rủi ro lỗi cao: công thức tính điểm, quy đổi hệ 4, xếp loại, kiểm tra ngày sinh, thao tác danh sách liên kết, tìm kiếm, sắp xếp, xóa node và chặn điểm ngoài khoảng.

Bảng 5.1: Các tình huống kiểm thử tự động

TC	Mục tiêu	Dữ liệu kiểm thử	Kết quả mong đợi
1	Tính điểm tổng theo trọng số	QT=8, GK=7, CK=9	Tổng = 8.20
2	Quy đổi điểm hệ 10 sang hệ 4	8.7	Hệ 4 = 3.7
3	Xếp loại học lực	TB10 = 8.1	Giỏi
4	Kiểm tra ngày sinh	2006-05-19 và 19-05-2006	Đúng và sai tương ứng
5	Thêm node và đếm số lượng	Thêm 2 sinh viên	Count = 2
6	Tìm kiếm tuyến tính	MSSV 202400002	Tìm thấy
7	Sắp xếp theo tên	B Student, A Student	A Student đứng đầu
8	Xóa node	Xóa MSSV 202400001	Xóa thành công
9	Thêm điểm hợp lệ	SV, môn và lớp đều tồn tại	Trả về true

TC	Mục tiêu	Dữ liệu kiểm thử	Kết quả mong đợi
10	Thêm điểm môn thứ hai	Hai bản ghi khác môn	Tính được trung bình
11	Tính trung bình có tín chỉ	MI1010 3TC, IT1110 2TC	TB10 = 8.40
12	Chặn điểm sai	Điểm quá trình = 11	Trả về false

5.3.2 Kết quả kiểm thử

Kết quả chạy `make test` cho thấy toàn bộ 12 ca kiểm thử đều đạt. Điều này chứng minh các hàm tính toán và thao tác cấu trúc dữ liệu chính hoạt động đúng với dữ liệu mẫu.



```

Kết quả kiểm thử tự động
[PASS] Tính điểm tổng = 20% QT + 30% GK + 50% CK
[PASS] Quy đổi điểm hệ 10 sang hệ 4
[PASS] Xếp loại học lực giỏi
[PASS] Kiểm tra danh sách ngày sinh
[PASS] Danh sách liên kết addBack và count
[PASS] Tìm kiếm tuyển tình theo MSSV
[PASS] Sắp xếp nội bộ theo tên sinh viên
[PASS] Xóa node trong danh sách liên kết
[PASS] Thêm điểm học tập
[PASS] Thêm điểm môn thứ hai
[PASS] Tính điểm trung bình có trọng số tín chỉ
[PASS] Chặn điểm ngoại khoảng 0..10

Tổng kết test: 12 PASS, 0 FAIL

```

Hình 5.8: Kết quả kiểm thử tự động

5.4 Kiểm thử thủ công qua giao diện

Bên cạnh kiểm thử tự động, nhóm thực hiện kiểm thử thủ công bằng cách nhập các lựa chọn qua menu. Mục tiêu là bảo đảm các chức năng tích hợp không lỗi khi chạy liên tục.

Bảng 5.2: Tổng hợp kiểm thử thủ công qua menu

Mã	Tác vụ	Cách kiểm tra	Kết quả
M1	Mở menu	Chạy <code>./student_manager</code>	Đạt
M2	Xem sinh viên	Chọn 1, kiểm tra bảng không lệch cột	Đạt
M3	Xem bảng điểm cá nhân	Chọn 7, nhập 202419014	Đạt
M4	Xem bảng điểm lớp HP	Chọn 8, nhập MI3310-01	Đạt
M5	Nhập điểm hợp lệ	Chọn 6, nhập điểm trong khoảng 0..10	Đạt
M6	Nhập điểm sai	Nhập điểm lớn hơn 10	Bị chặn
M7	Sắp xếp sinh viên	Chọn 9, chọn tiêu chí tên	Đạt
M8	In danh sách thi	Chọn 10, nhập MI3310-01	Đạt
M9	Xuất GUI	Chọn 11	Tạo file HTML
M10	Lưu dữ liệu	Chọn 0	File được ghi lại

5.5 Đánh giá chất lượng chương trình

Sau quá trình kiểm thử, chương trình đạt các tiêu chí chính: biên dịch thành công, chạy được các chức năng cốt lõi, lưu được dữ liệu vào file, không dùng cấu trúc dữ liệu nâng cao bị cấm, có kiểm thử tự động và có giao diện trình bày dữ liệu. Những điểm có thể phát triển thêm gồm hỗ trợ nhập dữ liệu trực tiếp trên Dashboard, xuất PDF danh sách thi, phân quyền người dùng và kiểm tra ngày sinh kỹ hơn theo từng tháng.

CHƯƠNG 6.

TỔNG KẾT

6.1 Các kỹ thuật đã vận dụng

Trong quá trình thực hiện đề tài, nhóm đã vận dụng nhiều kỹ thuật của môn Kỹ thuật lập trình. Những kỹ thuật này không chỉ xuất hiện trong mã nguồn mà còn được kiểm chứng qua kết quả chạy và kiểm thử.

Bảng 6.1: Tổng kết các kỹ thuật đã vận dụng

Kỹ thuật	Cách vận dụng trong chương trình	Module liên quan
Con trỏ và cấp phát động	Tạo node bằng <code>new</code> , nối node bằng <code>next</code> , giải phóng bằng <code>delete</code> .	<code>LinkedList.h</code>
Danh sách liên kết đơn	Lưu sinh viên, môn học, lớp học phần và điểm.	<code>LinkedList.h</code>
Tìm kiếm tuyến tính	Tìm sinh viên, môn học, lớp học phần và bản ghi điểm.	<code>Services.cpp</code>
Sắp xếp nổi bọt	Sắp xếp sinh viên theo MSSV hoặc họ tên.	<code>LinkedList.h</code>
Xử lý file text	Đọc ghi các file phân tách bằng dấu <code> </code> .	<code>DataStore.cpp</code>
Tách module	Chia chương trình thành dữ liệu, lưu trữ, nghiệp vụ, giao diện, GUI, test.	Toàn bộ dự án
Kiểm tra dữ liệu	Kiểm tra ngày sinh, điểm 0..10, mã sinh viên/môn/lớp tồn tại.	<code>Services.cpp</code> , <code>Ui.cpp</code>
Kiểm thử tự động	Viết <code>test_runner.cpp</code> với 12 ca kiểm thử.	<code>tests</code>
Giao diện CLI	Menu số, bảng dữ liệu căn cột, báo cáo in.	<code>Ui.cpp</code>

Kỹ thuật	Cách vận dụng trong chương trình	Module liên quan
Xuất GUI HTML	Sinh Dashboard HTML từ dữ liệu trong RAM.	GuiGenerator.cpp

6.2 Mức độ đáp ứng yêu cầu đề bài

Đối chiếu với yêu cầu chung, chương trình đã đáp ứng các điểm chính. Chương trình dùng C++, có menu chọn tác vụ đến khi kết thúc, dữ liệu được nhập và lưu trong file text, cấu trúc dữ liệu và thuật toán được tự cài đặt, có kiểm thử và có báo cáo kết quả thực hiện bằng hình ảnh. Đề tài cũng bám sát chủ đề quản lý sinh viên: quản lý danh sách sinh viên, môn học, lớp học phần, nhập điểm, tính GPA, xếp loại, tìm kiếm bảng điểm, in bảng điểm lớp học phần và danh sách thi.

Bảng 6.2: Đối chiếu sản phẩm với yêu cầu đề bài

Yêu cầu	Cách đáp ứng	Trạng thái
Dùng C/C++/C#/Java/Python	Sản phẩm viết bằng C++ chuẩn C++11.	Đạt
Không dùng cấu trúc nâng cao có sẵn	Tự cài <code>LinkedList</code> , tự tìm kiếm và sắp xếp.	Đạt
Có menu thao tác	Menu gồm 11 chức năng và lựa chọn 0 để thoát.	Đạt
Dữ liệu vào ra lưu file text	Dùng 4 file trong thư mục <code>data</code> .	Đạt
Quản lý sinh viên, môn học	Có file, struct và chức năng hiển thị/quản lý.	Đạt
Quản lý điểm	Có nhập/cập nhật điểm theo MSSV và mã môn.	Đạt
Tính GPA và xếp loại	Có hàm tính TB10, GPA4 và xếp loại.	Đạt
Tìm kiếm và báo cáo	Có bảng điểm cá nhân, lớp học phần, danh sách thi.	Đạt
Kiểm thử	Có <code>test_runner.cpp</code> và ảnh kết quả 12 PASS.	Đạt
Giao diện	Có CLI và Dashboard HTML.	Đạt

6.3 Hạn chế và hướng phát triển

Mặc dù chương trình đã hoàn thiện các chức năng chính, vẫn còn một số hạn chế. Thứ nhất, giao diện nhập liệu trực tiếp vẫn là CLI; Dashboard HTML hiện chủ yếu dùng để hiển thị. Thứ hai, chương trình chưa có đăng nhập và phân quyền giữa giảng viên, sinh viên, cán bộ quản lý. Thứ ba, kiểm tra ngày sinh mới dùng ở định dạng ký tự, chưa kiểm tra sâu số ngày trong từng tháng. Thứ tư, dữ liệu file text phù hợp với quy mô nhỏ, nhưng nếu số lượng sinh viên rất lớn thì cần cơ sở dữ liệu để truy vấn nhanh hơn.

Trong tương lai, nhóm có thể phát triển thêm các chức năng: nhập dữ liệu từ file CSV, xuất báo cáo PDF, bổ sung giao diện web có form nhập điểm, thêm chức năng thống kê tỷ lệ sinh viên đạt/không đạt, lưu lịch sử cập nhật điểm và phân quyền người dùng. Nếu có thời gian, nhóm cũng có thể tối ưu danh sách liên kết bằng cách thêm con trỏ `tail` để thao tác thêm cuối đạt $O(1)$.

6.4 Kết luận

Đề tài quản lý sinh viên và điểm số đã giúp nhóm vận dụng tổng hợp kiến thức của học phần Kỹ thuật lập trình vào một sản phẩm hoàn chỉnh. Nhóm đã phân tích nghiệp vụ, thiết kế cấu trúc dữ liệu, tự cài đặt danh sách liên kết đơn, tổ chức file dữ liệu, xây dựng các hàm xử lý điểm, tạo giao diện menu, xuất Dashboard HTML và viết kiểm thử tự động. Sản phẩm có thể biên dịch và chạy, có dữ liệu mẫu, có hình ảnh minh chứng và có báo cáo LaTeX đi kèm.

Thông qua đề tài, nhóm hiểu rõ hơn vai trò của thiết kế trước khi lập trình. Nếu không tách dữ liệu, nghiệp vụ, giao diện và file I/O thành các module riêng, chương trình rất dễ rối và khó sửa. Ngược lại, khi có cấu trúc rõ ràng, việc thêm chức năng như xuất Dashboard HTML hoặc bổ sung kiểm thử trở nên thuận tiện hơn. Đây là kinh nghiệm quan trọng cho các học phần lập trình và phát triển phần mềm sau này.

TÀI LIỆU THAM KHẢO

1. Bài giảng học phần Kỹ thuật lập trình MI3310, Đại học Bách khoa Hà Nội.
2. Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley.
3. Cplusplus.com. Tài liệu tham khảo cú pháp C++ cơ bản: input/output stream, file stream và string.
4. Tài liệu hướng dẫn GNU Make và trình biên dịch g++.

CHƯƠNG A.

PHÂN CÔNG CÔNG VIỆC

Bảng A.1: Bảng phân công nhiệm vụ và khối lượng công việc

Thành viên	MSSV	Nhiệm vụ chính	Tỷ lệ
Đào Văn Tuyến	202419014	Phân tích yêu cầu, thiết kế cấu trúc dữ liệu, cài đặt <code>LinkedList</code> , <code>Services</code> , chức năng nhập/cập nhật điểm, tính GPA, sắp xếp và kiểm thử.	55%
Lê Nguyễn Hữu Nghĩa	202418954	Thiết kế file dữ liệu, cài đặt đọc/ghi file, giao diện CLI, Dashboard HTML, chuẩn bị dữ liệu mẫu, ảnh kết quả chạy và hoàn thiện báo cáo LaTeX.	45%

CHƯƠNG B.

CẤU TRÚC THƯ MỤC DỰ ÁN

B.1 Cây thư mục bàn giao



Hình B.1: Cây thư mục bàn giao của dự án

B.2 Mô tả các thư mục

Bảng B.1: Mô tả các thư mục trong gói nộp

Thư mục/file	Mô tả
include/	Chứa các file header khai báo struct, template danh sách liên kết, hàm nghiệp vụ, hàm đọc/ghi file, giao diện và xuất GUI.
src/	Chứa file nguồn C++ triển khai các module.
data/	Chứa dữ liệu mẫu để chạy chương trình.

Thư mục/file	Mô tả
<code>gui/</code>	Chứa file <code>dashboard.html</code> được sinh từ chương trình.
<code>tests/</code>	Chứa <code>test_runner.cpp</code> để kiểm thử tự động.
<code>docs/figures/</code>	Chứa ảnh chụp kết quả chạy và sơ đồ đưa vào báo cáo.
<code>latex/</code>	Chứa <code>main.tex</code> và file PDF biên dịch từ LaTeX.
<code>logs/</code>	Chứa transcript đầu ra của các lần chạy chương trình.
<code>Makefile</code>	Tự động hóa biên dịch, chạy test và xuất GUI.
<code>README.md</code>	Hướng dẫn biên dịch, chạy chương trình và chạy kiểm thử.

CHƯƠNG C.

PHỤ LỤC MÃ NGUỒN CHÍNH

C.1 File main.cpp

```
1 #include "../include/Ui.h"
2
3 int main() {
4     runProgram();
5     return 0;
6 }
```

Listing C.1: Nội dung file main.cpp

C.2 File Models.h

```
1 #ifndef MODELS_H
2 #define MODELS_H
3
4 #include <string>
5 using std::string;
6
7 struct Student {
8     string id;
9     string fullName;
10    string className;
11    string birthDate;
12    string gender;
13    string phone;
14    string email;
15 };
16
17 struct Course {
18     string id;
19     string name;
20     int credits;
21 };
```

```
22
23 struct ClassSection {
24     string id;
25     string courseId;
26     string semester;
27     string academicYear;
28     string lecturer;
29 };
30
31 struct Grade {
32     string id;
33     string studentId;
34     string courseId;
35     string sectionId;
36     double processScore;
37     double midtermScore;
38     double finalScore;
39     double total10;
40 };
41
42 #endif
```

Listing C.2: Nội dung file Models.h

C.3 File LinkedList.h

```
1 #ifndef LINKED_LIST_H
2 #define LINKED_LIST_H
3
4 #include <string>
5 using std::string;
6
7 template <class T>
8 struct Node {
9     T data;
10    Node<T>* next;
11    Node(const T& value) {
12        data = value;
13        next = 0;
14    }
15 };
16
17 template <class T>
18 class LinkedList {
19 private:
20     Node<T>* head;
```

```

21
22 public:
23     LinkedList() { head = 0; }
24
25     ~LinkedList() { clear(); }
26
27     Node<T>* getHead() const { return head; }
28
29     bool isEmpty() const { return head == 0; }
30
31     int count() const {
32         int n = 0;
33         Node<T>* cur = head;
34         while (cur != 0) {
35             n++;
36             cur = cur->next;
37         }
38         return n;
39     }
40
41     void addBack(const T& value) {
42         Node<T>* node = new Node<T>(value);
43         if (head == 0) {
44             head = node;
45             return;
46         }
47         Node<T>* cur = head;
48         while (cur->next != 0) cur = cur->next;
49         cur->next = node;
50     }
51
52     T* findById(const string& id) const {
53         Node<T>* cur = head;
54         while (cur != 0) {
55             if (cur->data.id == id) return &(cur->data);
56             cur = cur->next;
57         }
58         return 0;
59     }
60
61     bool removeById(const string& id) {
62         if (head == 0) return false;
63         if (head->data.id == id) {
64             Node<T>* temp = head;
65             head = head->next;
66             delete temp;
67             return true;

```



```
68     }
69     Node<T>* prev = head;
70     Node<T>* cur = head->next;
71     while (cur != 0) {
72         if (cur->data.id == id) {
73             prev->next = cur->next;
74             delete cur;
75             return true;
76         }
77         prev = cur;
78         cur = cur->next;
79     }
80     return false;
81 }
82
83 void clear() {
84     Node<T>* cur = head;
85     while (cur != 0) {
86         Node<T>* nextNode = cur->next;
87         delete cur;
88         cur = nextNode;
89     }
90     head = 0;
91 }
92
93 void bubbleSort(int (*compare)(const T&, const T&)) {
94     if (head == 0 || head->next == 0) return;
95     bool swapped;
96     do {
97         swapped = false;
98         Node<T>* cur = head;
99         while (cur->next != 0) {
100             if (compare(cur->data, cur->next->data) > 0) {
101                 T tmp = cur->data;
102                 cur->data = cur->next->data;
103                 cur->next->data = tmp;
104                 swapped = true;
105             }
106             cur = cur->next;
107         }
108     } while (swapped);
109 }
110 };
111
112 #endif
```

Listing C.3: Nội dung file LinkedList.h

C.4 Các khai báo hàm nghiệp vụ trong Services.h

```
1 #ifndef SERVICES_H
2 #define SERVICES_H
3
4 #include "LinkedList.h"
5 #include "Models.h"
6 #include <string>
7 using std::string;
8
9 double round2(double x);
10 double calculateTotal10(double process, double midterm, double finalScore);
11 double convertToScale4(double score10);
12 string classifyByAverage10(double avg10);
13
14 Course* findCourse(LinkedList<Course>& courses, const string& courseId);
15 Student* findStudent(LinkedList<Student>& students, const string& studentId
16 );
17 ClassSection* findSection(LinkedList<ClassSection>& sections, const string&
18 sectionId);
19 Grade* findGrade(LinkedList<Grade>& grades, const string& studentId, const
20 string& courseId);
21
22
23 int countCreditsOfStudent(LinkedList<Grade>& grades, LinkedList<Course>&
24 courses, const string& studentId);
25 double calculateAverage10(LinkedList<Grade>& grades, LinkedList<Course>&
26 courses, const string& studentId);
27 double calculateGPA4(LinkedList<Grade>& grades, LinkedList<Course>& courses
28 , const string& studentId);
29
30 void updateAllTotalScores(LinkedList<Grade>& grades);
31 void sortStudentsById(LinkedList<Student>& students);
32 void sortStudentsByName(LinkedList<Student>& students);
33 void sortGradesByTotalDesc(LinkedList<Grade>& grades);
34
35 string makeGradeId(const string& studentId, const string& courseId);
36 bool addOrUpdateGrade(LinkedList<Grade>& grades,
37     LinkedList<Student>& students,
38     LinkedList<Course>& courses,
39     LinkedList<ClassSection>& sections,
40     const string& studentId,
41     const string& courseId,
42     const string& sectionId,
43     double process,
44     double midterm,
45     double finalScore,
46     string& message);
```

```
40  
41 bool validateDate(const string& date);  
42 bool validateScore(double score);  
43  
44 #endif
```

Listing C.4: Nội dung file Services.h

CHƯƠNG D.

PHỤ LỤC MÔ TẢ CÁC HÀM XỬ LÝ TÁC NGHIỆP CHÍNH

D.1 Nhóm hàm tính điểm

Bảng D.1: Mô tả nhóm hàm tính điểm

Hàm	Mô tả
calculateTotal10	Nhận ba điểm thành phần và trả về điểm tổng hệ 10 sau khi làm tròn hai chữ số thập phân.
convertToScale4	Quy đổi điểm hệ 10 sang hệ 4 theo thang điểm định nghĩa trong chương trình.
classifyByAverage10	Trả về chuỗi xếp loại học lực dựa trên điểm trung bình hệ 10.
calculateAverage10	Tính trung bình hệ 10 có trọng số tín chỉ của một sinh viên.
calculateGPA4	Tính GPA hệ 4 có trọng số tín chỉ của một sinh viên.

D.2 Nhóm hàm quản lý điểm

Bảng D.2: Mô tả nhóm hàm quản lý điểm

Hàm	Mô tả
findGrade	Duyệt danh sách điểm và tìm bản ghi có cùng MSSV và mã môn học.
makeGradeId	Sinh mã điểm theo dạng G_MSSV_MAMON.

Hàm	Mô tả
<code>validateScore</code>	Kiểm tra điểm nằm trong khoảng 0 đến 10.
<code>addOrUpdateGrade</code>	Kiểm tra toàn bộ ràng buộc dữ liệu, sau đó thêm mới hoặc cập nhật bản ghi điểm.
<code>updateAllTotalScores</code>	Duyệt toàn bộ danh sách điểm và tính lại điểm tổng.

D.3 Nhóm hàm báo cáo

Bảng D.3: Mô tả nhóm hàm báo cáo trong `Ui.cpp`

Hàm	Mô tả
<code>printStudents</code>	In danh sách sinh viên, số tín chỉ, điểm TB10, GPA4 và xếp loại.
<code>printCourses</code>	In danh mục môn học và số tín chỉ.
<code>printTranscriptOfStudent</code>	In bảng điểm cá nhân của sinh viên theo MSSV.
<code>printSectionScoreboard</code>	In bảng điểm của một lớp học phần.
<code>printExamSignatureReport</code>	In báo cáo danh sách thi có cột ký tên.
<code>printMainMenu</code>	Hiển thị menu chính cho người dùng lựa chọn.

CHƯƠNG E.

PHỤ LỤC TRÍCH MÃ NGUỒN XỬ LÝ FILE VÀ GUI

E.1 Trích mã nguồn đọc/ghi file

```
1 #include "../include/DataStore.h"
2 #include <fstream>
3 #include <iostream>
4 #include <cstdlib>
5 #include <sys/stat.h>
6 using namespace std;
7
8 static void ensureDataDir() {
9     #ifdef _WIN32
10         system("if not exist data mkdir data");
11     #else
12         mkdir("data", 0777);
13     #endif
14 }
15
16 int splitPipe(const string& line, string parts[], int maxParts) {
17     int count = 0;
18     string current = "";
19     for (int i = 0; i < (int)line.length(); i++) {
20         if (line[i] == '|') {
21             if (count < maxParts) parts[count++] = current;
22             current = "";
23         } else {
24             current += line[i];
25         }
26     }
27     if (count < maxParts) parts[count++] = current;
28     return count;
29 }
30
31 string safeField(string s) {
32     for (int i = 0; i < (int)s.length(); i++) {
33         if (s[i] == '|') s[i] = '/';
```

```

34     if (s[i] == '\n' || s[i] == '\r') s[i] = ' ';
35 }
36 return s;
37 }
38
39 void loadStudents(LinkedList<Student>& students, const string& filePath) {
40     ifstream fin(filePath.c_str());
41     if (!fin.is_open()) return;
42     string line;
43     while (getline(fin, line)) {
44         if (line.length() == 0) continue;
45         string p[10];
46         if (splitPipe(line, p, 10) >= 7) {
47             Student s;
48             s.id = p[0]; s.fullName = p[1]; s.className = p[2];
49             s.birthDate = p[3]; s.gender = p[4]; s.phone = p[5]; s.email = p
               [6];
50             students.addBack(s);
51         }
52     }
53     fin.close();
54 }
55
56 void loadCourses(LinkedList<Course>& courses, const string& filePath) {
57     ifstream fin(filePath.c_str());
58     if (!fin.is_open()) return;
59     string line;
60     while (getline(fin, line)) {
61         if (line.length() == 0) continue;
62         string p[6];
63         if (splitPipe(line, p, 6) >= 3) {
64             Course c;
65             c.id = p[0]; c.name = p[1]; c.credits = atoi(p[2].c_str());
66             courses.addBack(c);
67         }
68     }
69     fin.close();
70 }
71
72 void loadSections(LinkedList<ClassSection>& sections, const string&
    filePath) {
73     ifstream fin(filePath.c_str());
74     if (!fin.is_open()) return;
75     string line;
76     while (getline(fin, line)) {
77         if (line.length() == 0) continue;
78         string p[8];

```

```

79     if (splitPipe(line, p, 8) >= 5) {
80         ClassSection sec;
81         sec.id = p[0]; sec.courseId = p[1]; sec.semester = p[2];
82         sec.academicYear = p[3]; sec.lecturer = p[4];
83         sections.addBack(sec);
84     }
85 }
86 fin.close();
87 }
88
89 void loadGrades(LinkedList<Grade>& grades, const string& filePath) {
90     ifstream fin(filePath.c_str());
91     if (!fin.is_open()) return;
92     string line;
93     while (getline(fin, line)) {
94         if (line.length() == 0) continue;
95         string p[12];
96         if (splitPipe(line, p, 12) >= 8) {
97             Grade g;
98             g.id = p[0]; g.studentId = p[1]; g.courseId = p[2]; g.sectionId =
                p[3];
99             g.processScore = atof(p[4].c_str());
100            g.midtermScore = atof(p[5].c_str());
101            g.finalScore = atof(p[6].c_str());
102            g.total10 = atof(p[7].c_str());
103            grades.addBack(g);
104        }
105    }
106    fin.close();
107 }
108
109 bool saveStudents(const LinkedList<Student>& students, const string&
    filePath) {
110     ensureDataDir();
111     ofstream fout(filePath.c_str());
112     if (!fout.is_open()) return false;
113     Node<Student>* cur = students.getHead();
114     while (cur != 0) {
115         Student s = cur->data;
116         fout << safeField(s.id) << "|" << safeField(s.fullName) << "|" <<
            safeField(s.className) << "|"
117             << safeField(s.birthDate) << "|" << safeField(s.gender) << "|" <<
            safeField(s.phone) << "|"
118             << safeField(s.email) << "\n";
119         cur = cur->next;
120     }
121     fout.close();

```



```

122     return true;
123 }
124
125 bool saveCourses(const LinkedList<Course>& courses, const string& filePath)
126 {
127     ensureDataDir();
128     ofstream fout(filePath.c_str());
129     if (!fout.is_open()) return false;
130     Node<Course>* cur = courses.getHead();
131     while (cur != 0) {
132         Course c = cur->data;
133         fout << safeField(c.id) << "|" << safeField(c.name) << "|" << c.
134             credits << "\n";
135         cur = cur->next;
136     }
137     fout.close();
138     return true;
139 }
140
141 bool saveSections(const LinkedList<ClassSection>& sections, const string&
142     filePath) {
143     ensureDataDir();
144     ofstream fout(filePath.c_str());
145     if (!fout.is_open()) return false;
146     Node<ClassSection>* cur = sections.getHead();
147     while (cur != 0) {
148         ClassSection s = cur->data;
149         fout << safeField(s.id) << "|" << safeField(s.courseId) << "|" <<
150             safeField(s.semester) << "|" <<
151             << safeField(s.academicYear) << "|" << safeField(s.lecturer) << "
152                 \n";
153         cur = cur->next;
154     }
155     fout.close();
156     return true;
157 }
158
159 bool saveGrades(const LinkedList<Grade>& grades, const string& filePath) {
160     ensureDataDir();
161     ofstream fout(filePath.c_str());
162     if (!fout.is_open()) return false;
163     Node<Grade>* cur = grades.getHead();
164     while (cur != 0) {
165         Grade g = cur->data;
166         fout << safeField(g.id) << "|" << safeField(g.studentId) << "|" <<
167             safeField(g.courseId) << "|" <<

```

```

162         << safeField(g.sectionId) << "|" << g.processScore << "|" << g.
            midtermScore << "|"
163         << g.finalScore << "|" << g.total10 << "\n";
164         cur = cur->next;
165     }
166     fout.close();
167     return true;
168 }
169
170 void loadAll(LinkedList<Student>& students,
171             LinkedList<Course>& courses,
172             LinkedList<ClassSection>& sections,
173             LinkedList<Grade>& grades) {
174     students.clear(); courses.clear(); sections.clear(); grades.clear();
175     loadStudents(students, "data/students.txt");
176     loadCourses(courses, "data/courses.txt");
177     loadSections(sections, "data/sections.txt");
178     loadGrades(grades, "data/grades.txt");
179 }
180
181 bool saveAll(const LinkedList<Student>& students,
182             const LinkedList<Course>& courses,
183             const LinkedList<ClassSection>& sections,
184             const LinkedList<Grade>& grades) {
185     bool ok1 = saveStudents(students, "data/students.txt");
186     bool ok2 = saveCourses(courses, "data/courses.txt");
187     bool ok3 = saveSections(sections, "data/sections.txt");
188     bool ok4 = saveGrades(grades, "data/grades.txt");
189     return ok1 && ok2 && ok3 && ok4;
190 }

```

Listing E.1: Một phần nội dung file DataStore.cpp

E.2 Trích mã nguồn xử lý nghiệp vụ

```

1 #include "../include/Services.h"
2 #include <cmath>
3 #include <sstream>
4 using namespace std;
5
6 double round2(double x) {
7     return floor(x * 100.0 + 0.5) / 100.0;
8 }
9
10 double calculateTotal10(double process, double midterm, double finalScore)
    {

```

```
11     return round2(process * 0.2 + midterm * 0.3 + finalScore * 0.5);
12 }
13
14 double convertToScale4(double score10) {
15     if (score10 >= 9.0) return 4.0;
16     if (score10 >= 8.5) return 3.7;
17     if (score10 >= 8.0) return 3.5;
18     if (score10 >= 7.0) return 3.0;
19     if (score10 >= 6.5) return 2.5;
20     if (score10 >= 5.5) return 2.0;
21     if (score10 >= 5.0) return 1.5;
22     if (score10 >= 4.0) return 1.0;
23     return 0.0;
24 }
25
26 string classifyByAverage10(double avg10) {
27     if (avg10 >= 9.0) return "Xuat sac";
28     if (avg10 >= 8.0) return "Gioi";
29     if (avg10 >= 6.5) return "Kha";
30     if (avg10 >= 5.0) return "Trung binh";
31     if (avg10 >= 4.0) return "Yeu";
32     return "Kem";
33 }
34
35 Course* findCourse(LinkedList<Course>& courses, const string& courseId) {
36     return courses.findById(courseId);
37 }
38
39 Student* findStudent(LinkedList<Student>& students, const string& studentId
40 ) {
41     return students.findById(studentId);
42 }
43
44 ClassSection* findSection(LinkedList<ClassSection>& sections, const string&
45     sectionId) {
46     return sections.findById(sectionId);
47 }
48
49 Grade* findGrade(LinkedList<Grade>& grades, const string& studentId, const
50     string& courseId) {
51     Node<Grade>* cur = grades.getHead();
52     while (cur != 0) {
53         if (cur->data.studentId == studentId && cur->data.courseId ==
54             courseId) return &(cur->data);
55         cur = cur->next;
56     }
57     return 0;
58 }
```

```

54 }
55
56 int countCreditsOfStudent(LinkedList<Grade>& grades, LinkedList<Course>&
    courses, const string& studentId) {
57     int total = 0;
58     Node<Grade>* cur = grades.getHead();
59     while (cur != 0) {
60         if (cur->data.studentId == studentId) {
61             Course* c = findCourse(courses, cur->data.courseId);
62             if (c != 0) total += c->credits;
63         }
64         cur = cur->next;
65     }
66     return total;
67 }
68
69 double calculateAverage10(LinkedList<Grade>& grades, LinkedList<Course>&
    courses, const string& studentId) {
70     double sum = 0;
71     int credits = 0;
72     Node<Grade>* cur = grades.getHead();
73     while (cur != 0) {
74         if (cur->data.studentId == studentId) {
75             Course* c = findCourse(courses, cur->data.courseId);
76             if (c != 0) {
77                 sum += cur->data.total10 * c->credits;
78                 credits += c->credits;
79             }
80         }
81         cur = cur->next;
82     }
83     if (credits == 0) return 0;
84     return round2(sum / credits);
85 }
86
87 double calculateGPA4(LinkedList<Grade>& grades, LinkedList<Course>& courses
    , const string& studentId) {
88     double sum = 0;
89     int credits = 0;
90     Node<Grade>* cur = grades.getHead();
91     while (cur != 0) {
92         if (cur->data.studentId == studentId) {
93             Course* c = findCourse(courses, cur->data.courseId);
94             if (c != 0) {
95                 sum += convertToScale4(cur->data.total10) * c->credits;
96                 credits += c->credits;
97             }

```

```

98     }
99     cur = cur->next;
100 }
101 if (credits == 0) return 0;
102 return round2(sum / credits);
103 }
104
105 void updateAllTotalScores(LinkedList<Grade>& grades) {
106     Node<Grade>* cur = grades.getHead();
107     while (cur != 0) {
108         cur->data.total10 = calculateTotal10(cur->data.processScore, cur->
            data.midtermScore, cur->data.finalScore);
109         cur = cur->next;
110     }
111 }
112
113 static int cmpStudentId(const Student& a, const Student& b) {
114     if (a.id < b.id) return -1;
115     if (a.id > b.id) return 1;
116     return 0;
117 }
118
119 static int cmpStudentName(const Student& a, const Student& b) {
120     if (a.fullName < b.fullName) return -1;
121     if (a.fullName > b.fullName) return 1;
122     return 0;
123 }
124
125 static int cmpGradeTotalDesc(const Grade& a, const Grade& b) {
126     if (a.total10 > b.total10) return -1;
127     if (a.total10 < b.total10) return 1;
128     return 0;
129 }
130
131 void sortStudentsById(LinkedList<Student>& students) { students.bubbleSort(
    cmpStudentId); }
132 void sortStudentsByName(LinkedList<Student>& students) { students.
    bubbleSort(cmpStudentName); }
133 void sortGradesByTotalDesc(LinkedList<Grade>& grades) { grades.bubbleSort(
    cmpGradeTotalDesc); }
134
135 string makeGradeId(const string& studentId, const string& courseId) {
136     return "G_" + studentId + "_" + courseId;
137 }
138
139 bool validateScore(double score) {
140     return score >= 0.0 && score <= 10.0;

```

```

141 }
142
143 bool validateDate(const string& date) {
144     if (date.length() != 10) return false;
145     if (date[4] != '-' || date[7] != '-') return false;
146     for (int i = 0; i < 10; i++) {
147         if (i == 4 || i == 7) continue;
148         if (date[i] < '0' || date[i] > '9') return false;
149     }
150     return true;
151 }
152
153 bool addOrUpdateGrade(LinkedList<Grade>& grades,
154                     LinkedList<Student>& students,
155                     LinkedList<Course>& courses,
156                     LinkedList<ClassSection>& sections,
157                     const string& studentId,
158                     const string& courseId,
159                     const string& sectionId,
160                     double process,
161                     double midterm,
162                     double finalScore,
163                     string& message) {
164     if (findStudent(students, studentId) == 0) {
165         message = "Khong ton tai ma sinh vien.";
166         return false;
167     }
168     if (findCourse(courses, courseId) == 0) {
169         message = "Khong ton tai ma mon hoc.";
170         return false;
171     }
172     if (findSection(sections, sectionId) == 0) {
173         message = "Khong ton tai lop hoc phan.";
174         return false;
175     }
176     if (!validateScore(process) || !validateScore(midterm) || !validateScore(
177         finalScore)) {
178         message = "Diem phai nam trong khoang 0..10.";
179         return false;
180     }
181     Grade* existed = findGrade(grades, studentId, courseId);
182     if (existed != 0) {
183         existed->sectionId = sectionId;
184         existed->processScore = process;
185         existed->midtermScore = midterm;
186         existed->finalScore = finalScore;
187         existed->total10 = calculateTotal10(process, midterm, finalScore);

```

```

187     message = "Cap nhat diem thanh cong.";
188     return true;
189 }
190 Grade g;
191 g.id = makeGradeId(studentId, courseId);
192 g.studentId = studentId;
193 g.courseId = courseId;
194 g.sectionId = sectionId;
195 g.processScore = process;
196 g.midtermScore = midterm;
197 g.finalScore = finalScore;
198 g.total10 = calculateTotal10(process, midterm, finalScore);
199 grades.addBack(g);
200 message = "Them diem thanh cong.";
201 return true;
202 }

```

Listing E.2: Một phần nội dung file Services.cpp

E.3 Trích mã nguồn xuất Dashboard GUI

```

1  #include "../include/GuiGenerator.h"
2  #include "../include/Services.h"
3  #include <fstream>
4  #include <iomanip>
5  #include <sstream>
6  #include <cstdlib>
7  #include <sys/stat.h>
8  using namespace std;
9
10 static void ensureGuiDir() {
11     #ifdef _WIN32
12         system("if not exist gui mkdir gui");
13     #else
14         mkdir("gui", 0777);
15     #endif
16 }
17
18 static string htmlEscape(string s) {
19     string out = "";
20     for (int i = 0; i < (int)s.length(); i++) {
21         char c = s[i];
22         if (c == '&') out += "&amp;";
23         else if (c == '<') out += "&lt;";
24         else if (c == '>') out += "&gt;";
25         else if (c == '"') out += "&quot;";

```

```

26     else out += c;
27 }
28 return out;
29 }
30
31 static string fmt2(double x) {
32     ostringstream oss;
33     oss << fixed << setprecision(2) << x;
34     return oss.str();
35 }
36
37 bool exportGuiDashboard(LinkedList<Student>& students,
38                         LinkedList<Course>& courses,
39                         LinkedList<ClassSection>& sections,
40                         LinkedList<Grade>& grades,
41                         const string& outputPath) {
42     ensureGuiDir();
43     ofstream f(outputPath.c_str());
44     if (!f.is_open()) return false;
45
46     int studentCount = students.count();
47     int courseCount = courses.count();
48     int sectionCount = sections.count();
49     int gradeCount = grades.count();
50     double sumAvg = 0.0;
51     int avgN = 0;
52     Node<Student>* sCur = students.getHead();
53     while (sCur != 0) {
54         double avg = calculateAverage10(grades, courses, sCur->data.id);
55         if (avg > 0) { sumAvg += avg; avgN++; }
56         sCur = sCur->next;
57     }
58     double avgSystem = avgN == 0 ? 0.0 : round2(sumAvg / avgN);
59
60     f << "<!DOCTYPE html><html lang='vi'><head><meta charset='UTF-8'>";
61     f << "<meta name='viewport' content='width=device-width, initial-scale=1.0'>";
62     f << "<title>Quan ly sinh vien va diem so</title>";
63     f << "<style>";
64     f << "/*{box-sizing:border-box}body{margin:0;font-family:'Segoe UI',Arial,sans-serif;background:#eef3f8;color:#17202a;}";
65     f << ".layout{display:flex;min-height:100vh}.sidebar{width:285px;background:#1f2d3d;color:white;padding:28px 22px;}";
66     f << ".brand{font-size:22px;font-weight:700;margin-bottom:8px}.sub{font-size:13px;color:#b8c7d9;margin-bottom:30px}.nav div{padding:13px 15px;border-radius:12px;margin:8px 0;background:#30455f}.nav div.active{background:#2f80ed;}";

```



```

67 f << ".main{flex:1;padding:30px 34px}.top{display:flex;justify-content:
    space-between;align-items:center;margin-bottom:22px}.top h1{margin:0;
    font-size:30px}.badge{background:#eaf3ff;color:#0b4a8b;padding:10px
    16px;border-radius:999px;font-weight:600}.cards{display:grid;grid-
    template-columns:repeat(5,1fr);gap:18px;margin-bottom:22px}.card{
    background:white;border-radius:18px;padding:22px;box-shadow:0 10px 25
    px rgba(31,45,61,.08)}.num{font-size:34px;font-weight:800;color:#1
    f2d3d}.lab{color:#607089;margin-top:5px}.panel{background:white;
    border-radius:20px;padding:22px;margin-bottom:22px;box-shadow:0 10px
    25px rgba(31,45,61,.08)}";
68 f << "table{width:100%;border-collapse:collapse;font-size:14px}th{
    background:#f3f7fb;text-align:left;color:#34495e}th,td{padding:11px
    10px;border-bottom:1px solid #e6ebf2}.pill{padding:5px 10px;border-
    radius:999px;font-weight:600;font-size:12px}.good{background:#eaf8ed;
    color:#126b34}.warn{background:#fff7e6;color:#8a5a00}.bad{background
    :#fff0f0;color:#a42222}.grid2{display:grid;grid-template-columns:1fr
    1fr;gap:18px}.footer{font-size:13px;color:#6b778d;text-align:right;
    margin-top:18px}";
69 f << "</style></head><body><div class='layout'><aside class='sidebar'><
    div class='brand'>QL Sinh viên</div><div class='sub'>C++ Project -
    MI3310 KTLT</div><div class='nav'><div class='active'>Dashboard</div>
    <div>Danh sách sinh viên</div><div>Môn ọc</div><div>ỚLp ọc ầphn</
    div><div>ậNhập điểm</div><div>ảNg điểm</div><div>Danh sách thi</div>
    </div></aside><main class='main'>";
70 f << "<div class='top'><h1>Dashboard ảnun lý sinh viên và điểm ố</h1><
    div class='badge'>ữD ệliu ừt file text</div></div>";
71 f << "<section class='cards'>";
72 f << "<div class='card'><div class='num'>" << studentCount << "</div><
    div class='lab'>Sinh viên</div></div>";
73 f << "<div class='card'><div class='num'>" << courseCount << "</div><div
    class='lab'>Môn ọc</div></div>";
74 f << "<div class='card'><div class='num'>" << sectionCount << "</div><
    div class='lab'>ỚLp ọc ầphn</div></div>";
75 f << "<div class='card'><div class='num'>" << gradeCount << "</div><div
    class='lab'>ảBn ghi điểm</div></div>";
76 f << "<div class='card'><div class='num'>" << fmt2(avgSystem) << "</div>
    <div class='labĐ'>ểim TB ệh 10</div></div>";
77 f << "</section>";
78
79 f << "<section class='panel'><h2>Danh sách sinh viên và GPA</h2><table><
    tr><th>MSSV</th><th>QH tên</th><th>ỚLp</th><th>Tín ịch</th><th>TB10</
    th><th>GPA4</th><th>ếXp ặloại</th></tr>";
80 sCur = students.getHead();
81 while (sCur != 0) {
82     Student s = sCur->data;
83     double avg10 = calculateAverage10(grades, courses, s.id);
84     double gpa4 = calculateGPA4(grades, courses, s.id);

```

```

85     string cls = avg10 >= 8 ? "good" : (avg10 >= 5 ? "warn" : "bad");
86     f << "<tr><td>" << htmlEscape(s.id) << "</td><td>" << htmlEscape(s.
        fullName) << "</td><td>" << htmlEscape(s.className) << "</td><td>"
        << countCreditsOfStudent(grades, courses, s.id) << "</td><td>" <<
        fmt2(avg10) << "</td><td>" << fmt2(gpa4) << "</td><td><span class
        ='pill " << cls << ">" << htmlEscape(classifyByAverage10(avg10))
        << "</span></td></tr>";
87     sCur = sCur->next;
88 }
89 f << "</table></section>";
90
91 f << "<section class='grid2'><div class='panel'><h2>Ảng đđim ởlp MI3310
    -01</h2><table><tr><th>MSSV</th><th>QH tên</th><th>QT</th><th>GK</th>
    <th>CK</th><th>ỔTng</th></tr>";
92 Node<Grade>* gCur = grades.getHead();
93 while (gCur != 0) {
94     if (gCur->data.sectionId == "MI3310-01") {
95         Student* s = findStudent(students, gCur->data.studentId);
96         f << "<tr><td>" << htmlEscape(gCur->data.studentId) << "</td><td>"
            << htmlEscape(s ? s->fullName : "Khong ro") << "</td><td>" <<
            fmt2(gCur->data.processScore) << "</td><td>" << fmt2(gCur->data
            .midtermScore) << "</td><td>" << fmt2(gCur->data.finalScore) <<
            "</td><td>" << fmt2(gCur->data.total10) << "</td></tr>";
97     }
98     gCur = gCur->next;
99 }
100 f << "</table></div><div class='panel'><h2>ỂKim soát ệnghip ựv</h2><
    table><tr><th>Ràng ộbuc</th><th>ạTrng thái</th></tr>";
101 f << "<tr><td>Không ậnhp đđim ngoài ảkhong 0..10</td><td><span class='
    pill goodĐ'>ang ậbt</span></td></tr>";
102 f << "<tr><td>Không trùng MSSV khi thêm ởmi</td><td><span class='pill
    goodĐ'>ang ậbt</span></td></tr>";
103 f << "<tr><td>ựT tính đđim ổtng 20% - 30% - 50%</td><td><span class='
    pill good'>ựT động</span></td></tr>";
104 f << "<tr><td>Đợc/ghi file text ặng pipe-separated</td><td><span class
    ='pill good'>ặHot động</span></td></tr>";
105 f << "</table></div></section>";
106 f << "<div class='footer'>Sinh ởbi module GuiGenerator.cpp - có ểth ởm
    ựtrc ếtip ầbng trình ệduyt.</div>";
107 f << "</main></div></body></html>";
108 f.close();
109 return true;
110 }

```

Listing E.3: Nội dung file GuiGenerator.cpp

CHƯƠNG F.

PHỤ LỤC KIỂM THỬ

F.1 File `test_runner.cpp`

```
1 #include "../include/LinkedList.h"
2 #include "../include/Models.h"
3 #include "../include/Services.h"
4 #include <iostream>
5 #include <cmath>
6 using namespace std;
7
8 static bool almost(double a, double b) { return fabs(a - b) < 0.001; }
9
10 int main() {
11     int passed = 0, failed = 0;
12     auto check = [&](bool cond, const char* name) {
13         if (cond) { cout << "[PASS] " << name << "\n"; passed++; }
14         else { cout << "[FAIL] " << name << "\n"; failed++; }
15     };
16
17     check(almost(calculateTotal10(8.0, 7.0, 9.0), 8.2), "Tinh diem tong =
18         20% QT + 30% GK + 50% CK");
19     check(almost(convertToScale4(8.7), 3.7), "Quy doi diem he 10 sang he 4")
20         ;
21     check(classifyByAverage10(8.1) == "Gioi", "Xep loai hoc luc gioi");
22     check(validateDate("2006-05-19") == true && validateDate("19-05-2006")
23         == false, "Kiem tra dinh dang ngay sinh");
24
25     LinkedList<Student> students;
26     Student a; a.id = "202400001"; a.fullName = "B Student"; a.className = "
27         IT1"; a.birthDate = "2006-01-01"; a.gender = "Nam";
28     Student b; b.id = "202400002"; b.fullName = "A Student"; b.className = "
29         IT1"; b.birthDate = "2006-01-02"; b.gender = "Nu";
30     students.addBack(a); students.addBack(b);
31     check(students.count() == 2, "Danh sach lien ket addBack va count");
32     check(students.findById("202400002") != 0, "Tim kiem tuyen tinh theo
33         MSSV");
```

```

28  sortStudentsByName(students);
29  check(students.getHead()->data.fullName == "A Student", "Sap xep noi bot
    theo ten sinh vien");
30  check(students.removeById("202400001") == true && students.count() == 1,
    "Xoa node trong danh sach lien ket");
31
32  LinkedList<Course> courses;
33  Course c1; c1.id = "MI1010"; c1.name = "Giai tich 1"; c1.credits = 3;
34  Course c2; c2.id = "IT1110"; c2.name = "Tin hoc dai cuong"; c2.credits =
    2;
35  courses.addBack(c1); courses.addBack(c2);
36
37  LinkedList<ClassSection> sections;
38  ClassSection sec1; sec1.id = "MI1010-01"; sec1.courseId = "MI1010"; sec1
    .semester = "2025.1"; sec1.academicYear = "2025-2026"; sec1.lecturer
    = "TS. A";
39  ClassSection sec2; sec2.id = "IT1110-01"; sec2.courseId = "IT1110"; sec2
    .semester = "2025.1"; sec2.academicYear = "2025-2026"; sec2.lecturer
    = "TS. B";
40  sections.addBack(sec1); sections.addBack(sec2);
41
42  LinkedList<Grade> grades;
43  string msg;
44  check(addOrUpdateGrade(grades, students, courses, sections, "202400002",
    "MI1010", "MI1010-01", 8, 8, 8, msg), "Them diem hop le");
45  check(addOrUpdateGrade(grades, students, courses, sections, "202400002",
    "IT1110", "IT1110-01", 9, 9, 9, msg), "Them diem mon thu hai");
46  check(almost(calculateAverage10(grades, courses, "202400002"), 8.4), "
    Tinh diem trung binh co trong so tin chi");
47  check(!addOrUpdateGrade(grades, students, courses, sections, "202400002"
    , "MI1010", "MI1010-01", 11, 8, 8, msg), "Chan diem ngoai khoang
    0..10");
48
49  cout << "\nTong ket test: " << passed << " PASS, " << failed << " FAIL\n
    ";
50  return failed == 0 ? 0 : 1;
51 }

```

Listing F.1: Nội dung file kiểm thử tự động

F.2 Hướng dẫn chạy lại kiểm thử

Người chấm có thể chạy lại kiểm thử bằng các lệnh sau trong thư mục gốc của dự án:

```
make clean  
make  
make test
```

Nếu kết quả hiển thị 12 PASS, 0 FAIL, chương trình đã vượt qua toàn bộ các kiểm thử tự động nhóm thiết kế.